

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHASSIST
13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

This thesis documents the development of DASH-01, a custom biped with a parallel-linkage leg. The robot asks how fast a small, low-cost legged robot can run. More centrally, it asks what transferring reinforcement-learning locomotion methods onto real hardware actually costs.

The work covers the full chain. It models a leg that closes a kinematic loop, simulates that model in MuJoCo, and runs a measurement campaign that replaces every assumed plant parameter with a measured one. It develops three successive action spaces for the learned controller: direct torque, per-step position targets, and a central pattern generator tuned by the policy. It adds a curriculum that releases the base degrees of freedom one at a time. It ends with the deployment stack on the robot itself: a 200 Hz control daemon on a non-realtime Raspberry Pi, a web-based bring-up instrument, and an always-on flight recorder.

The headline simulation result is a policy that runs at 2.6 m/s on the fully free plant without falling. The headline measurement result is less comfortable. The real robot is 18 % heavier and about 15 % weaker than the plant every policy trained against, and its actuator position loop is more than an order of magnitude slower. Those corrections move the robot's credible ceiling to roughly 2.7 m/s burst and 1.3 m/s sustained. DASH-01 is therefore a walking and jogging demonstrator, not a record platform.

The thesis argues, with evidence, that such a gap is the normal product of simulation-first development. The countermeasures are methodological: measure early, treat every training aid and every reward term as an exploit target, evaluate on fixed tasks with paired seeds, and instrument the hardware so that failures stay reconstructible. A hand-designed classical controller, written late in the project as a baseline, beats the learned policy on the constrained plant. It then fails at exactly the same wall on the free plant. The binding constraint is the robot, not the learning algorithm.

Table of Contents

Abstract	2
1 Introduction	6
1.1 Problem statement	6
1.2 Objectives of the Project	6
1.3 Methodology Overview	6
1.4 Contributions	6
2 Background and State of the Art	8
2.1 What makes bipedal locomotion hard	8
2.2 A short history: six answers to one question	9
2.2.1 Stability from geometry: static walking and the zero-moment point	9
2.2.2 Stability from the mechanism: passive dynamics	9
2.2.3 Stability from elasticity: spring-mass templates	10
2.2.4 Stability from foot placement: the hopping lineage	10
2.2.5 Stability from a provable limit cycle: hybrid zero dynamics	11
2.2.6 Stability from re-solving the problem: optimisation-based control	11
2.2.7 Stability from experience: learned control	12
2.2.8 Summary	13
2.3 The hopping lineage's assumptions, and why DASH-01 breaks all three	14
2.4 The actuation axis, which decides which approach is available	14
2.5 Control approaches: where this thesis sits	15
2.5.1 Two kinds of gait prior	15
2.5.2 What learning should buy in stability margin	16
2.6 RL for locomotion: the lineage of this architecture	17
2.7 Existing platforms, and where DASH-01 sits	17
3 Development Methodology	19
3.1 Overall methodology	19
3.2 Hardware selection	19
3.2.1 Motor	19
3.2.2 What the actuator selection got wrong	22
3.2.3 Battery pack	23
3.2.4 Compute	27
3.2.5 Sensors	27
3.3 Robot modelling	29
3.4 Simulation	30
3.4.1 URDF cannot describe this robot	30
3.4.2 Compute backend	30
3.4.3 How do you know a simulation result is real?	31
3.5 Plant identification: replacing assumptions with measurements	31
3.5.1 Mass	32
3.5.2 Peak torque is what the gearbox delivers, not what the datasheet quotes	32
3.5.3 The drive's position loop is 0.8 Hz	33
3.5.4 The ankle spring	34
3.5.5 System identification: including what it could not resolve	34
3.5.6 Loop timing and CAN	35
3.5.7 The CAN protocol, as measured	35

3.6	Achievable gait: actuator and mass-distribution limits	36
3.6.1	Method	36
3.6.2	Results	36
3.6.3	Limiting factors	37
3.6.4	Sensitivity to mass distribution	37
3.6.5	Ankle stiffness	37
3.6.6	The corrected ceiling	38
3.6.7	Limitations	38
3.7	Toy problem: one joint, end to end	38
3.8	RL policy design and validation	39
3.8.1	Attempt 1: direct joint torque	39
3.8.2	Attempt 2: joint position targets (PD)	40
3.8.3	Attempt 3: a central pattern generator tuned by the policy	41
3.8.4	Reflexes: what is learned and what is hard-wired	43
3.8.5	The pitch-balance wall, and how it was broken	45
3.8.6	The cadence problem and a cautionary bug	47
3.8.7	Results	48
3.8.8	Which of these results survive the plant correction	49
3.8.9	Validation status and known gaps	49
3.8.10	A second generator: explicit-oscillator CPG versus the Fourier series	50
3.8.11	The ankle, revisited as a build decision	51
3.8.12	The drivable-demonstrator lineage: closing on the measured plant	52
3.8.13	A scripted baseline: how far does no learning get?	54
3.8.14	Postscript: the rebuilt robot, and a cap that was never there	56
3.9	Negative results	57
3.10	Deployment	58
3.10.1	Linux without a real-time kernel	58
3.10.2	CAN at runtime: the bus with nothing on it	59
3.10.3	The drive as deployed	60
3.10.4	The safety envelope as an instrument	60
3.10.5	Hardware validation: the flight recorder and the pre-move guard	61
3.11	Hardware experiments	63
3.11.1	Static end-effector force map	64
4	Results	65
4.1	Hardware	65
4.2	RL experiments	65
5	Discussion	66
5.1	Successes	66
5.2	Failures	66
5.3	Lessons learned	66
6	Conclusion and Future Work	68
6.1	Summary of Findings	68
6.2	Challenges and Limitations	68
6.3	Suggestions for Future Research	68
7	Appendix	70
7.1	logbook	70
7.2	URDF from CAD: step by step	70

References	71
------------	----

1 Introduction

1.1 Problem statement

Reinforcement learning has produced striking legged-locomotion results on a small set of well-instrumented research platforms. Reproducing those results on a new, custom, imperfectly known robot is a different problem, and it is the problem most robot builders actually face. On such a robot the plant parameters are uncertain, the actuators hide undocumented internal control loops, the sensors are commodity parts, and the computer is a single board. The gap between “RL walks in simulation” and “RL walks on this robot” is dominated by modelling fidelity, actuation bandwidth, calibration and instrumentation, not by learning algorithms.

DASH-01 is a two-legged, six-motor robot with a parallel-linkage knee and a passive ankle, designed around the ambition of high running speed. This thesis takes the robot from a partially finished mechanical design to a measured, simulated, trained and instrumented platform. It reports what each stage cost and what each stage revealed, and it includes the negative results deliberately.

1.2 Objectives of the Project

1. Model the robot faithfully enough to train against: kinematics, including the closed 4-bar loop, plus masses, actuators and sensors.
2. Establish the robot’s physical performance envelope independently of any controller, so that learned results can be judged against a ceiling rather than against hope.
3. Train locomotion policies with reinforcement learning. Iterate on the action-space design until one design converges reliably, then quantify what still separates those policies from hardware deployment.
4. Build the deployment stack on the real robot — control loop, calibration, safety and telemetry — and validate as much of the chain as the hardware state allows.
5. Record the methodology honestly enough to be reproducible, including the failures.

1.3 Methodology Overview

The project starts halfway. The mechanical design existed in part, so the work reported here begins at modelling and at hardware selection for the non-mechanical subsystems. The development order was as follows. First, CAD re-assembly and export to a simulable model. Second, validation of that model against the physical robot, by workspace calibration, then statics, then full dynamic identification. Third, a first-principles capability analysis on the validated model. Fourth, a toy problem — a single-joint inverted pendulum — to exercise the complete train–export–deploy chain at minimum stakes. Fifth, three generations of RL policy design, trained on an HPC cluster. Last, the deployment stack on the robot, driven by a measurement campaign that fed corrections back into the simulation. Section 3.1 details the sequence. The single most consequential methodological finding is that *every* assumed plant parameter that was later measured moved in the pessimistic direction (Table 3.7).

1.4 Contributions

- A complete CAD-to-MJCF modelling pipeline for a biped whose linkage closes a loop. It includes the loop-closure treatment that URDF cannot express, and validation tooling: analytic

Jacobians checked against finite differences, and loop closure checked against the measured backdriven workspace at 100 % of 2789 poses.

- A measured plant. It comprises weighed segment masses, torque constants identified by a known-mass method that breaks the degeneracy between K_t and gravity, the Bode response of the drive’s internal position loop (0.8 Hz plus 25 ms), IMU noise with provenance labels, loop timing and the CAN protocol. It states explicitly what the identification could *not* resolve.
- A first-principles capability analysis of running speed, validated against MuJoCo. Its sensitivity result — distal mass costs roughly nine times what torso mass costs — was confirmed repeatedly by later measurement as the binding design axis.
- A policy architecture and the curriculum that trains it. The architecture is a learned Fourier gait generator, re-parameterised every step and mirrored across legs, with hard-wired reflexes, learned reflexes and a per-step residual channel. The curriculum releases the base degrees of freedom one at a time. An A/B against an explicit-oscillator generator ran on identical plants and rewards.
- A catalogue of failure modes with measured diagnoses. It covers reward exploits (skating, floor penetration, shuttling and suicide by penalty), curriculum deadlocks, evaluation restore bugs, a warm-start bug that silently duplicated every seed replicate, and the pitch-balance wall traced to plant statics rather than to learning.
- A scripted classical baseline: a capture point and a clocked gait, 21 gains, no learning. It outperforms RL on the constrained plant and fails identically on the free plant, which is evidence that the wall is physical.
- Deployment infrastructure on the robot: a 200 Hz daemon on stock Linux, a web bring-up instrument that carried most of the measurement campaign, and a three-tier flight recorder with a pre-move safety guard. The recorder was built in response to a hardware failure and validated by reproducing that failure.
- Two published, reusable component databases, each with figure-of-merit tooling. The actuator explorer holds 140 commercial actuators, 121 motors, 121 gearboxes and 9 drives across eight vendors, and is built on the winding-invariant motor constant $K_m = K_t/\sqrt{R}$ with explicit electrical reference frames. The cell and pack explorer holds roughly 430 cells, 54 BMS modules and 110 packs, and is built on duration-stamped current tiers and a pack synthesiser. Both audited this robot’s own hardware choices after the fact (Section 3.2.2). The audit vindicates the actuator selection as the Pareto-optimal choice in its class. It also shows that the battery was selected against a peak current rating quoted without a duration.

2 Background and State of the Art

2.1 What makes bipedal locomotion hard

A legged robot is not a robot arm with feet. An arm is bolted to the world, so the floor reacts every joint torque the arm produces through its base. A legged robot touches the world only through intermittent, unilateral contacts. Three consequences follow, and they organise this chapter.

It is underactuated. A biped with six leg motors has twelve degrees of freedom. The extra six belong to the body, which translates along three axes and rotates about three more with no motor attached to any of them. Those six *base* degrees of freedom can be influenced only indirectly, through the ground. A flight phase has no ground, so the centre of mass follows a ballistic arc that no actuation can alter. The robot can only choose the posture it will land in. This thesis makes the point structurally. It releases the base degrees of freedom one at a time as a training curriculum (Section 3.8.3), and the milestone at which the robot fails names the degree of freedom it cannot control.

The control inputs are unilateral and saturating. A foot can push on the ground but not pull. It can push only within a friction cone, or it slips. The point where that push applies is the *centre of pressure*, and it can move only within the contact patch. For a point foot the patch is a point, so the centre of pressure cannot move at all. The most powerful input available is *where to put the next foot*, and that input is not continuous. It is chosen once per step, and the leg's reachable workspace bounds it. A controller therefore holds a small number of discrete, bounded opportunities to act rather than continuous authority.

The timescales are short. An inverted pendulum of leg length h diverges with time constant $\sqrt{h/g}$, which is about 310 ms for this robot. Ground contacts in running last tens of milliseconds. Any correction must therefore be planned, commanded and *mechanically executed* inside a window of a few tenths of a second. Actuator bandwidth is consequently a first-class constraint, not an implementation detail. This project measured that constraint the hard way: the drive's position loop is a 0.8 Hz pole with $\tau = 199$ ms, against the 310 ms divergence (Section 3.5.3).

Running adds one thing to walking. In walking the centre of mass *vaults* over a stiff support leg and rises at mid-stance. In running the centre of mass *bounces*: it falls at mid-stance as the leg compresses, rises again as the leg rebounds, and passes through an aerial phase with both feet off the ground. The distinction is not cosmetic. It changes which reduced model applies, which is the subject of Section 2.2.3. It is also why frameworks built for walking do not extend to running when the commanded speed is simply raised.

The full dynamics are intractable to reason about directly, so the field works with *reduced models*: the smallest system that reproduces the behaviour of interest. Three such models appear repeatedly in this thesis. The **inverted pendulum**, a point mass on a rigid massless leg, models walking and gives the divergence time above. The **spring-loaded inverted pendulum** (SLIP), a point mass on a springy leg, models running. The **capture point** [24] is the location at which a foot must land to bring the inverted pendulum exactly to rest, and is therefore the natural target for a foot-placement controller. It underlies both the hand-wired pitch reflex of Section 3.8.4 and the scripted baseline of Section 3.8.13. These toy models stay load-bearing on a real robot, and not as a convenient simplification: the free-topple time of the plain inverted pendulum turns out to *be* the failure signature this thesis spends a chapter diagnosing (Table 3.17).

2.2 A short history: six answers to one question

The history of walking robots is more legible when it is read as a sequence of answers to a single question: **where does the stability come from?** Every family below answers that question differently, and every answer buys something and costs something. The six families are not a progression from worse to better. All six are in active use, and the choice among them is governed less by fashion than by what the hardware can physically do, which is the subject of Section 2.4.

2.2.1 Stability from geometry: static walking and the zero-moment point

The first walking robots avoided the dynamics rather than solving them. WABOT-1 was built at Waseda University in 1973 and is generally credited as the first full-scale anthropomorphic walker. It moved slowly enough that its own momentum could be neglected [19]. It was, in effect, a statue that occasionally shifted its weight. Stability meant keeping the vertical projection of the centre of mass inside the *support polygon*, the convex hull of the points touching the ground. A robot that satisfies that condition at every instant cannot tip, by a purely geometric argument that needs no model of the dynamics.

The dynamic generalisation is the **zero-moment point** (ZMP). Vukobratović and Juricic introduced it in 1969 [35] and later surveyed it themselves [34]. The ZMP is the point on the ground about which the horizontal components of the ground reaction moment vanish. Equivalently, it is the point at which gravity and all inertial forces can be replaced by a single vertical force. While the ZMP stays strictly inside the support polygon, the foot does not rotate about its edge and the contact behaves as though the robot were bolted to the floor. One scalar condition thus converts the balance problem into a geometric constraint that a planner can check.

The framework carried the field for three decades. Honda's P2 and its descendant ASIMO [9] made it famous. Kajita's 3D linear inverted pendulum model [15] and ZMP preview control [14] made it cheap. Both reduce the robot to a point mass constrained to a plane of constant height, then solve for a centre-of-mass trajectory whose ZMP tracks a planned footstep sequence. That problem is small enough to solve online on 1990s computers.

What it buys. Transparency and a checkable guarantee. The condition is a single inequality and needs no contact model beyond the geometry of the foot. The foot is never permitted to rotate, so the approach tolerates a fairly inaccurate dynamic model. It is safe, and for a long time it was the only approach that worked.

What it costs. The guarantee is bought by never leaving the regime in which it holds. The support polygon *is* the foot, so the robot needs large flat feet and flat ground. Holding the centre of mass at constant height demands a permanently bent-knee posture, which is why ASIMO-class walkers show a cost of transport near 1.6 against roughly 0.2 for a walking human. Toe-off and heel-strike are the foot rotations that make human walking efficient, and the framework forbids both by construction, because both place the ZMP on the boundary. Most decisively for this thesis, ZMP is a *walking* framework. Running contains an interval with no support polygon at all, so the central condition is not merely violated but undefined.

2.2.2 Stability from the mechanism: passive dynamics

The opposite extreme arrived with McGeer's passive dynamic walkers [20]. These are a pair of legs with no motors, no sensors and no computer, and they walk stably down a shallow ramp on gravity alone. The gait is a *limit cycle*: a closed orbit in state space that the robot returns to on its own after a small disturbance. That orbit is a property of the linkage's geometry and mass distribution, not of any controller. Collins, Ruina, Tedrake and Wisse [4] later added the minimum actuation needed to walk on the level, an ankle push-off replacing the slope. They

measured a cost of transport of about 0.20, comparable to a human and an order of magnitude better than the ZMP walkers of the same era.

What it buys. Efficiency that no other approach has matched, and a conceptual result that outlives the hardware: **morphology is control**. A well-proportioned leg performs most of the stabilisation for free. The actuators then fight nothing, because the commanded motion is the motion the mechanism already wanted.

What it costs. Authority. The basin of attraction is tiny, and a centimetre of unexpected terrain ends the gait. Speed and heading are essentially not commandable, and each walker is tuned to a single operating point. The result is stability without controllability.

The lesson that transfers is the one this thesis re-derives from the opposite direction. A mechanism that carries the gait needs only a weak controller. A mechanism that fights the gait cannot be rescued by any controller. Section 3.11.1 and Section 3.6 make that statement quantitative for DASH-01.

2.2.3 Stability from elasticity: spring-mass templates

Biomechanics supplied the model that walking frameworks lack. Blickhan [2] showed that one model describes running, hopping and trotting across animals spanning orders of magnitude in body mass. That model is the *spring-loaded inverted pendulum*: a point mass on a massless linear spring. The leg compresses on touchdown, stores the impact energy elastically, and returns it at push-off. Full and Koditschek [7] named such a model a *template*, meaning the simplest system that exhibits the behaviour, with the physical animal or robot as its *anchor*. They argued that good legged design consists of building an anchor that behaves like its template.

The engineering consequence is direct. A running robot wants a real spring in its leg. A large part of the controller's job is then to hold the mechanism near a spring-mass limit cycle, rather than to compute a trajectory from scratch. Hurst's ATRIAS [10] is the most literal instantiation: a biped built to *be* a SLIP, with fibreglass leaf springs in series with its drives. Cassie and Digit are its commercial descendants.

What it buys. Correct physics for running, rather than a walking framework pushed past its validity. The spring recovers impact energy that the gearbox would otherwise dissipate as heat. The spring also rejects high-frequency disturbances at the speed of sound in steel, which is faster than any feedback loop can react. That second property is often missed.

What it costs. The spring rate is a *design commitment*, chosen once, in hardware. It sets the leg's natural frequency and therefore the narrow band of speeds at which the robot is efficient, and software cannot retune it. Series elasticity also inserts a low-pass filter between motor and ground, which caps force-control bandwidth. Both costs appear in this thesis in their sharpest form. DASH-01's passive ankle spring is exactly such a commitment, and getting its stiffness wrong is the central mechanical finding of the work (Sections 3.8.11 and 3.11.1).

2.2.4 Stability from foot placement: the hopping lineage

Raibert's hopping robots [27] answered the question with the insight that reframed the field. A running robot does not have to be balanced *continuously*. It has to be balanced *once per step*, at the instant it commits to where the next foot will go. Control then decomposes into three nearly independent loops. Thrust energy injected at bottom of stance sets the hopping height. Foot placement relative to a neutral point sets the forward speed,

$$x_f = \frac{1}{2} \dot{x} T_s + k(\dot{x} - \dot{x}_d), \quad (2.1)$$

and hip torque applied *during stance only* corrects body attitude. Stance is when friction pins the foot, so the leg can serve as a reaction strut. The lineage runs from the 1984 one-legged hopper through BigDog [26] to the Boston Dynamics Atlas. It remains the skeleton of most classical running controllers, including the scripted baseline of Section 3.8.13, in which Equation (2.1) appears directly.

What it buys. Running, which is what ZMP cannot do. It needs no support polygon, so point feet are permissible. It needs no accurate dynamic model, because the placement law carries essentially one empirical gain. The three-loop decomposition is also simple enough to implement on 1980s hardware.

What it costs. Three assumptions, examined in Section 2.3 because DASH-01 violates all three. And no proof: the hoppers demonstrably worked, but the approach offers no certificate that they will.

2.2.5 Stability from a provable limit cycle: hybrid zero dynamics

Passive dynamics gives a limit cycle that cannot be commanded. Raibert gives commandable behaviour that cannot be proved. Hybrid zero dynamics (HZD) is the synthesis. The controller does not track a trajectory indexed by *time*. It enforces *virtual constraints* instead: relations among the joint angles parameterised by the robot’s own progress through the step. Enforcing those constraints collapses the closed-loop dynamics onto a low-dimensional surface, the *zero dynamics*, whose remaining motion has a single degree of freedom. A Poincaré map can then certify the orbital stability of the resulting gait. Westervelt, Grizzle and Koditschek [38] established the framework, the accompanying monograph [37] systematised it, and RABBIT [3] demonstrated it on a physical underactuated biped with point feet.

What it buys. The only genuine stability proof in this chapter, for an underactuated robot with point feet, which is precisely the case ZMP cannot address. It also buys an idea that has outlived its framework: indexing a gait by *phase* rather than by time. That idea reappears in essentially every modern learned locomotion architecture, including the gait phase φ of this thesis’s own generator (Section 3.8.3).

What it costs. The proof is a proof about the *model*. The gait itself comes from an offline trajectory optimisation over that model, so the accuracy requirements are severe and the guarantee degrades silently when the plant is wrong. This thesis met that failure mode in a different guise (Table 3.7). Each gait is a separate optimisation, so speed changes, turning and terrain must be handled by scheduling among precomputed cycles rather than emerging from the controller.

2.2.6 Stability from re-solving the problem: optimisation-based control

By the late 2000s computers were fast enough to solve the control problem online, repeatedly. Wieber [39] recast ZMP walking as *model-predictive control*. At each cycle the controller solves a constrained optimisation over a short future horizon, applies only the first input, discards the rest, and re-solves at the next cycle. The constraint set is written down explicitly, so friction cones, torque bounds and kinematic limits become constraints rather than hoped-for margins. Kuindersma et al. [17] built the whole-body version that ran Atlas at the DARPA Robotics Challenge. Di Carlo et al. [6] produced the convex formulation on MIT Cheetah 3 that now underlies most classical legged stacks.

What it buys. Constraints as first-class objects. Genuine feedback at the level of the trajectory rather than only at the joints. And the commandability that passive dynamics and HZD lack: one controller walks at different speeds, turns, and recovers from pushes.

What it costs. Three things. An accurate model, because the optimiser trusts it over the whole horizon and the errors compound. A solver fast enough to close a loop, which constrains the formulation to be convex or nearly so. And, the requirement that decides this thesis, **high-bandwidth torque control**, because the output is a force command that must reach the ground essentially unfiltered. Which foot contacts when is usually still prescribed by hand, because optimising the contact schedule turns a continuous problem into a combinatorial one.

2.2.7 Stability from experience: learned control

The most recent answer replaces the analytic chain with a function fitted in simulation. A policy, typically a small neural network, is optimised by reinforcement learning to maximise a reward over millions of simulated episodes. Planner, whole-body controller and joint tracker collapse into a single evaluation. Heess et al. [8] showed that rich terrain and a simple reward produce recognisable running gaits with no gait prior at all. PPO [28] became the workhorse algorithm, and fast simulators such as MuJoCo [33] made the sample cost affordable.

Transfer to hardware is the hard part. The toolkit that solved it is worth naming, because this thesis uses all of it. *Domain randomisation* [32, 31] varies the plant parameters every episode, so the policy is optimised over a *set* of robots and cannot overfit any single wrong one. *Learned actuator models* [11] replace the idealised motor with a network identified from hardware data, and that step first carried RL onto a real quadruped convincingly. *Teacher-student distillation* [18, 21] trains with privileged simulator state, then distils into a policy that observes only what the robot can actually sense. On bipeds, Siekmann et al. [30] demonstrated the full range of gaits on Cassie through periodic reward composition. Radosavovic et al. [25] carried the approach to a full humanoid. Crowley et al. [5] produced the current bipedal 100 m record: 24.73 s, or about 4.1 m/s average. Motion priors from reference data are a parallel strand [22], relevant here as the archetype of a *shape* prior (Section 2.5.1).

What it buys. No analytic contact model is required. That matters, because contact is exactly where analytic models are weakest and simulators are adequate. The policy is optimised over a distribution of states and plants rather than around one operating point, which is the source of its robustness claim (Section 2.5.2). Inference takes microseconds on cheap hardware, so the entire stack fits on a single-board computer.

What it costs. No stability proof, and no diagnosis when it fails. The reward function becomes the specification, and an optimiser satisfies the specification rather than the intent. Section 3.8.2 and Table 3.18 catalogue what that looks like in practice. The sample cost is 10^8 to 10^9 environment steps, so each result is a multi-day cluster job. And the whole result rests on the simulator, because every modelling error is silently baked into the policy. Section 3.5 is this document's demonstration of that last point.

2.2.8 Summary

Approach	Stability from	What it buys	What it costs, and what it demands of the hardware
Static / ZMP	Geometry: CoP inside the support polygon	Transparent, checkable, tolerant of model error, safe	Walking only — no flight phase; large flat feet and flat ground; poor efficiency (CoT ≈ 1.6). Position-controlled joints suffice.
Passive dynamics	The mechanism itself	Best-in-class efficiency (CoT ≈ 0.2); proves morphology is control	Almost no authority: tiny basin, fixed speed. Demands a carefully proportioned, low-inertia leg.
SLIP / series elasticity	Elastic energy storage	Correct model for running; recovers impact energy; sub-millisecond passive disturbance rejection	Spring rate is fixed in hardware, fixing the efficient speed band; caps force bandwidth. Demands a physical spring, sized correctly.
Foot placement (Raibert)	One decision per step	Running with point feet; near model-free; simple enough to hand-build	Three assumptions (Section 2.3); no proof. Demands actuators that execute a placement within one swing.
Hybrid zero dynamics	A certified limit cycle	Genuine stability proof under underactuation and point feet; phase-indexed gaits	Proof is against the model; one offline optimisation per gait. Demands an accurate model and good torque tracking.
MPC / whole-body	Re-solving online	Constraints first-class; commandable; unified planning and control	Model accuracy over the horizon; fast solver; contact schedule still prescribed. Demands high-bandwidth torque control.
Deep RL	Experience over a distribution	No contact model needed; robust by construction; runs on cheap compute	No proof; reward becomes the specification and is gamed; 10^8 – 10^9 samples; inherits every simulator error. Demands a faithful plant model — in software.

Table 2.1: The six families, read as answers to “where does the stability come from?”. The last column governs the choice in practice: an approach is available only if the actuation can meet its demand, which is the subject of Section 2.4.

Two observations close the survey. First, the families are complementary rather than competing, and most modern systems combine them. A typical modern system is a learned policy over a phase variable borrowed from HZD, with a spring borrowed from SLIP, on hardware descended from Raibert’s. This thesis is no exception (Section 2.5.1). Second, the trend across fifty years is a steady *transfer of the stability burden from the controller to the robot and to the model*. The burden moves first from a geometric condition checked at runtime to a mechanism that is stable on its own. It then moves to a simulator faithful enough that a policy trained in it survives reality. That trend is what makes measurement, rather than control design, the critical path of a

project like this one.

2.3 The hopping lineage’s assumptions, and why DASH-01 breaks all three

The three-loop decomposition of Section 2.2.4 is not free. It embeds three assumptions, and each is worth naming individually. DASH-01 violates all three, and the manner of each violation is a result of this thesis rather than a caveat to it.

1. **A point foot is sufficient, because attitude authority comes from the hip and not from the foot.** This assumption is the one most often quoted as a virtue of the approach, and it holds only while the hip route is open. Hip torque against a pinned foot is a reaction torque, so three conditions must hold. The leg needs a usable moment arm, the stance must last long enough to apply the torque, and the friction cone must hold. When any of those conditions fails, a point foot has *no* remaining attitude authority, because a point contact permits zero centre-of-pressure travel by definition. Section 3.8.13 measures exactly that state on DASH-01: at milestone m3 the single toe sphere yields zero centre-of-pressure authority at *any* ankle stiffness, so the ankle there is not weak but disconnected. Section 3.11.1 shows that the passive ankle which should have supplied the missing authority delivers 0.42 body weights against a 3.5 BW requirement, and clears that requirement over 0.0 % of the running-stance region. Two practical conclusions follow. A point foot is an acceptable simplification only for a robot whose hip-and-stance route is strong. And a foot with real fore-aft extent is a load-bearing design choice, not a refinement.
2. **The commanded foot placement is reachable.** Equation (2.1) returns a position, not a feasible one. On this robot the usable fore-aft workspace is ± 0.30 m of a 1.06 m geometric span (Section 3.6). The toe hangs about 0.92 m below the centre of mass, so a pitch rate $\dot{\theta}$ sweeps the contact point forward at $L\dot{\theta}$. That sweep was measured to outrun the controller’s corrective foot motion 2:1, until the workspace saturates in under a second (Section 3.8.13). The placement law does not fail because it is wrong. It fails because the plant diverges faster than the foot can travel, which is a statement about linkage proportions rather than about control.
3. **The actuator executes what the placement law commands, within one stance.** Raibert’s hydraulics did. DASH-01’s drive is a 0.8 Hz position loop with $\tau = 199$ ms (Section 3.5.3), against an inverted-pendulum divergence constant $\sqrt{h/g} = 311$ ms. The scripted baseline therefore latches the target once per swing rather than tracking it continuously, a structural choice Section 3.8.13 defends at length. Even so, the drive recovers only about 83 % of the commanded placement by touchdown.

Replacing the classical controller with a learned one repairs none of the three. That is the substance of Section 3.8.13, and the reason this thesis reaches a mechanical conclusion from a control experiment.

2.4 The actuation axis, which decides which approach is available

Table 2.1 ends each row with a hardware demand, and that column is not incidental. A control approach is available only to a robot whose actuators can meet its requirement. The actuation choice is therefore made *before* the control choice, whether or not the designer notices. Three families dominate.

Hydraulic. Raibert’s hoppers, BigDog [26] and the Boston Dynamics Atlas are hydraulic. The family offers enormous force density, very high bandwidth, and a commanded force that arrives

essentially instantaneously. The cost is a pump, a reservoir, noise, leaks and a floor on system size and budget that puts the family out of reach for most laboratories and all students.

Series elastic (SEA). Pratt and Williamson [23] proposed inserting a spring between gearbox and load deliberately. Force then becomes a *position* measurement, namely the spring’s deflection, which is cheap and accurate to sense. The spring also protects the gear teeth from impact and stores running energy in the SLIP sense of Section 2.2.3. ATRIAS, Cassie and Digit are the canonical bipeds [10]. The cost is that the spring is a low-pass filter. The spring-load resonance caps force bandwidth, and the compliance adds states that must themselves be controlled.

Quasi-direct drive (QDD), or proprioceptive actuation. The MIT Cheetah lineage [29, 36, 16] took the opposite route: a large-diameter, low- K_v motor behind a *small* gear ratio, typically 4:1 to 10:1, so that the transmission is nearly transparent in both directions. Three properties follow. Torque can be inferred from motor current with no torque sensor, which is why the family is called proprioceptive. Reflected inertia is low, so an impact backdrives the motor instead of shattering a gear train. And the current loop sets force bandwidth, which puts it in the kilohertz range. The costs are honest ones. Torque density is lower than a high-ratio gearbox gives, so the robot is heavier for a given torque. And torque is current, current is heat, so the binding limit becomes thermal rather than mechanical. This thesis meets that limit directly (Section 3.6.6).

The consequence for DASH-01 belongs in the background chapter rather than in the results. The robot’s actuators are QDD-class, chosen for exactly the reasons above, and Section 3.2.2 vindicates them as the Pareto-optimal choice in their class. They are nevertheless commanded through the vendor’s *position* loop, which Section 3.5.3 measures at 0.8 Hz. A position loop of that bandwidth wrapped around a transparent transmission discards the single property the transmission was chosen for. It also removes two entire rows of Table 2.1 from the menu. MPC and torque-mode RL both require that a commanded force reach the ground, and a 0.8 Hz position servo delivers neither. That is the mechanism behind the recommendation in Section 6.3: treat torque-mode control as a day-one architectural requirement rather than as an optimisation.

2.5 Control approaches: where this thesis sits

Classical stacks decompose the problem into a gait or footstep planner, a whole-body or centroidal controller, and joint-level tracking. They are transparent and analysable, but they demand accurate models and high-bandwidth torque control. Learned controllers collapse the same stack into a policy trained in simulation. They trade analysability for robustness to the messiness of contact, and they pay for it with a sim-to-real gap that modelling fidelity and domain randomisation must close. This thesis uses both. RL supplies the gait, and classical structure — reflexes, a clocked generator, capture-point priors — sits exactly where feedback must not be blind (Section 3.8.4). The scripted baseline of Section 3.8.13 is the classical counterfactual, built on the same plant to answer one question: was the learning ever the bottleneck?

2.5.1 Two kinds of gait prior

“Learned controllers need no gait prior” is a common and unhelpful summary, and this thesis is a poor advertisement for it. The architecture that finally worked here (Section 3.8.3) is a Fourier-series central pattern generator with hand-wired reflexes, which is a great deal of prior. The useful distinction is not *how much* prior but *which kind*. That distinction concerns the reachable set of gaits, not the amount of engineering involved.

A **shape prior** fixes the trajectory and exposes its timing. Raibert’s leg [27], a fixed limit cycle in the classical CPG tradition [12], a reference motion clip tracked by imitation [22] and any clocked foot-path generator all belong here. The controller chooses phase, frequency, amplitude and the

reset on contact, but the *shape* of the motion is authored in advance. The decisive property of a shape prior is that tuning does not enlarge what the robot can do. However carefully the gains are set, the reachable set of gaits stays a low-dimensional family around the authored gait. If the authored gait is wrong for the morphology, no amount of tuning reaches a better one.

A **structural prior** constrains the *class* of trajectories and leaves the member unspecified. The Fourier action space of Section 3.8.3 asserts only that the gait is periodic, smooth, bounded and mirrored left to right. Which periodic function it is remains a free parameter, and the reachable set grows monotonically with the harmonic count N , approaching every periodic gait the workspace admits. PMTG [13] and CPG-RL [1] sit here too. The prior does real work: it is the reason the problem is trainable at all, where the prior-free torque parameterisation of Section 3.8.1 never converged. But it works on the *search* rather than on the *answer*.

The distinction earns this much space because DASH-01 is precisely the case that separates the two. A shape prior is safe when the morphology has a natural template to copy. An animal with roughly these proportions can be filmed, and the observed gait is a defensible starting shape. This robot has no such template. Hardware calibration established that **the cam is a full crank**: it sweeps about 245° against a CAD-guessed $\pm 86^\circ$, and the 4-bar assembles at every angle (Section 3.11). The transmission ratio the 4-bar produces varies strongly with posture (Section 3.11.1), and no animal offers a gait for that. The cadence result is the evidence that the wider search space was genuinely used rather than merely available. Reinforcement learning repeatedly converged on a high-cadence solution, and the independent first-principles feasibility sweep of Section 3.6 located the unconstrained optimum at the same 12.4 steps/s from physics alone, with no learning in it. A learned gait found the plant’s own optimum on a morphology for which no template exists, and an independent analysis confirmed the location.

Two qualifications keep this honest. First, **periodicity alone is demonstrably not sufficient** on this plant. Section 2.6 reports the literature constraint that no published system runs on per-cycle-only gait updates, and an experiment reproduced it here. An ablation removed the per-step residual channel, leaving a pure oscillator, and produced stability without locomotion: 15 m of a 72 m course at 0.25 m/s (Section 3.8.10). The working policy class is periodic *plus* an aperiodic intra-cycle correction. The structural prior is an exploration scaffold, not the policy. Second, **the prior did not disappear, it moved**, and where it moved is informative. The sagittal pitch reflex is hand-tuned rather than learned (Section 3.8.4), and it sits deliberately at the one channel where feedback must not be blind. The design axis is therefore a dial, not a binary between prior and no prior. It runs from “which trajectory” to “which class of trajectories, plus explicit structure only where the plant demands it”.

2.5.2 What learning should buy in stability margin

The theoretical case for a learned controller on an unstable plant is a case about margin. A hand-designed controller is a local object. Its gains are tuned around an operating point, its structure encodes an assumed regime, and its region of attraction is whatever that construction happens to yield. A policy trained over a distribution of states and plants approximates, in the limit, a globally optimal feedback law over that distribution. Crucially, the hand-designed controller lies *inside* the policy class. With an exact plant and unbounded training, the learned optimum can therefore only match or exceed it.

The argument is correct, and its premises are load-bearing. That is why Section 3.8.13 finds the opposite on real numbers: a 21-gain scripted controller beats the learned reference on the constrained plant, and fails identically to it on the free one. Three premises fail. The plant is not exact. The training is not unbounded. And, least obviously and most importantly, a margin does not always exist to be won. Section 3.8.13 gives the measured breakdown. The conclusion

it supports is that the margin advantage of learning scales with how much margin the plant has to give. That is a statement about robots rather than about methods, and it is the reason this thesis ends in a mechanical recommendation.

2.6 RL for locomotion: the lineage of this architecture

Within the learned-control family of Section 2.2.7, the lineage relevant to this work is narrower. It is the family of *periodically parameterised* action spaces, in which the policy does not emit joint targets directly but shapes a periodic generator that does. Policies Modulating Trajectory Generators (PMTG) [13] established the pattern. Oscillator-based CPG-RL [1] instantiated it with coupled phase oscillators in the tradition of [12]. The clock and phase-reward methods of Siekmann et al. [30] carried Cassie to the 100 m Guinness record [5]. The appeal is exactly the structural prior of Section 2.5.1: the network no longer has to discover that locomotion is periodic, only which periodic signal it wants.

A literature review conducted mid-project (Section 3.8.3) returned one unambiguous constraint, and that constraint reshaped the architecture here. **No published system achieves running with per-cycle-only gait-parameter updates and no intra-cycle feedback channel.** Every successful system keeps a per-step path from observation to action. The same review supplied the phase-gated contact rewards and the efficiency terms adopted in Section 3.8.3. On the sim-to-real side, Section 3.8.12 exercises the standard toolkit surveyed in Section 2.2.7: domain randomisation with curricula, observation history in place of privileged state, and explicit sensor models. An ablation there shows which components actually carried the cost.

2.7 Existing platforms, and where DASH-01 sits

The bipeds that produce the results cited above fall into three groups, separated more by actuation and budget than by control philosophy.

Purpose-built research bipeds with series elasticity. ATRIAS [10] and its commercial descendants Cassie and Digit are built to be spring-mass systems. They carry leaf springs in series with the drives, a high-voltage bus that supports the joint speeds running demands, and a morphology deliberately unlike a human’s. They hold the bipedal speed record [5]. They are also the product of a decade of institutional effort, and are priced accordingly.

Quasi-direct-drive robots in the MIT Cheetah lineage [29, 36, 16]. These are predominantly quadrupeds, but the group is architecturally the most influential of the three. Essentially every recent legged platform has adopted the pattern it established, namely QDD actuation commanded in torque mode. That includes the current generation of commercial humanoids, which are QDD, torque-capable and RL-driven, and which have brought a capable biped within reach of a single laboratory’s budget for the first time. Section 6.3 concludes that DASH-01 should have adopted this architecture end to end.

Hobby-class humanoids, built from position-controlled smart servos. They are cheap and numerous, and they are restricted to the static-walking regime of Section 2.2.1 for a reason that is now familiar. A servo closing its own position loop cannot deliver a commanded ground force, so the dynamic families stay unavailable whatever controller is written above them.

DASH-01 sits deliberately between the second and third groups: QDD-class actuators and a running-oriented morphology, on a student budget and a single-board computer. Three design choices define it. A parallel 4-bar knee moves the knee actuator’s mass proximally. A passive ankle spring replaces a seventh and eighth motor. Six motors do the work of ten. Each choice is a bet that mass and cost can be traded against capability in the directions that matter for

speed. The sensitivity analysis of Table 3.11 confirms the first bet as the right instinct, at -0.93 (m/s)/kg for distal mass against -0.11 for the torso. Section 3.11.1 shows the second bet was not. What distinguishes this platform from its neighbours in the literature is not any single choice. It is that every choice has been *measured* rather than assumed, which is what makes the design recommendations of Section 6.3 evidence rather than opinion.

3 Development Methodology

3.1 Overall methodology

The project inherited a partially designed mechanical platform and proceeded in the following order, each stage gating the next:

1. **Modelling and hardware selection.** Re-assembly of the CAD, export to a simulable model (Section 3.3), and selection of the non-mechanical hardware: compute, sensing, battery (Section 3.2).
2. **Model validation.** Workspace calibration on the physical robot, then static validation, then — much later, with better instrumentation — full dynamic identification (Section 3.5). This stage carries the lesson of the project. It was done too late and too lightly at first, and every correction that followed moved the plant in the pessimistic direction.
3. **Capability analysis.** A first-principles gait-feasibility sweep on the validated model (Section 3.6), so that learned results have a ceiling to be judged against.
4. **Toy problem.** A single-motor inverted pendulum carrying the complete train–export–deploy loop (Section 3.7) before committing the 12-DOF robot to the same pipeline.
5. **RL policy development.** Three action-space generations, a reflex architecture, a base-DOF-release curriculum, and a generator A/B — trained on the EPFL Izar cluster (Section 3.8).
6. **Deployment.** The 200 Hz control stack, calibration and safety machinery, and the flight recorder on the robot itself (Section 3.10), plus the measurement campaign that fed the corrections of Section 3.5 back into every earlier stage.

Two cross-cutting disciplines proved more important than any single stage.

The first is *adversarial review of anything that produces a number*. Rewards, simulators, estimators and evaluators each produced a confident wrong answer before being caught (Sections 3.4.3 and 3.5.5).

The second is *measurement before assumption*. Three project-level decisions rested on assumed numbers: the abandonment of torque control, the choice of a 50 Hz control rate, and the question of whether the ankle needed an actuator. Later measurement reversed or reframed all three.

3.2 Hardware selection

3.2.1 Motor

The selection instrument. Actuator datasheets are not comparable as published. The choice was therefore made against a purpose-built component database and explorer, rather than against a spreadsheet of vendor headline figures.¹ The database holds 140 commercial integrated actuators, 121 motors, 121 gearboxes and 9 standalone drives, as four joined tables. Every unit is decomposed into its motor layer, its gearbox layer and its drive layer, so that a candidate can be *composed* rather than only looked up. The modelling follows Lee et al., *How to Model Brushless Electric Motors for the Design of Lightweight Robotic Systems* (arXiv:2310.00080).

Two ideas from that paper are what make a selection possible.

The motor constant is the only portable figure of merit. The motor constant is $K_m = K_t / \sqrt{R} = \tau / \sqrt{P_{\text{copper}}}$. It is invariant to operating current and to rewinding, because

¹<https://github.com/nathmo/actuatorReview>

rewinding n turns gives $K_t \propto n$ while $R \propto n^2$. Unlike K_t or K_v , it can therefore be compared across vendors. It also sets continuous torque through the thermal limit, as $\tau_{\text{cont}} = K_m \sqrt{\Delta T / R_{th}}$. Ranking candidates by K_v is the intuitive move, because K_v sets top speed, but it compares winding choices rather than motors.

Every electrical constant is meaningless without its reference frame. A K_t , K_v or R can be quoted phase, line-to-line, q-axis, bus or RMS, in wye or in delta, and each vendor picks a different convention. The same physical motor differs by up to $\sqrt{3}/2$, or by a factor of 3, depending on which convention is used. The database therefore stores a raw value plus an explicit frame tag, and normalises on the fly. Conversions that stay under-determined are flagged rather than guessed. The point is not pedantic: Section 3.2.2 shows that it lands directly on this project’s own K_t measurement.

Each leg carries three actuators. All six are CubeMars quasi-direct-drive units with an integrated drive that speaks a servo-mode CAN protocol.

	Hip roll (abduction)	Cam and thigh (sagittal)
Part	CubeMars AK60-39	CubeMars AKE90-8
Reduction	39:1	8:1
Peak torque (datasheet)	72 N·m	170 N·m
Continuous torque	24 N·m	55 N·m
No-load speed	10.3 rad/s	22 rad/s
Motor K_t (line-to-line)	0.12 N·m/A	0.272 N·m/A
Motor R (line-to-line)	0.60 Ω	0.164 Ω
Motor constant K_m	0.15 N·m/ $\sqrt{W}$	0.674 N·m/ $\sqrt{W}$
Motor current, cont./peak	4.5 / 17 A	21 / 72 A
Reflected armature	0.046 kg·m ²	0.0216 kg·m ²
Mass	0.75 kg	1.40 kg
Unit price	EUR 393	EUR 424

Table 3.1: The six actuators (two AK60-39, four AKE90-8), datasheet values as ingested into the actuator database. Section 3.5 measures the effective joint K_t at 82–89 % of these figures, which is consistent with about 85 % gearbox efficiency, so the deployed peak torques are 61.2 and 144.5 N·m respectively. See Section 3.2.2 for why that ratio may be partly a reference-frame artefact rather than gearbox loss.

The no-load speeds follow from the 48 V bus. The AK60-39 is a KV80 motor, so 80 rpm/V $\times$ 48 V/39 = 10.3 rad/s at the joint. The AKE90-8 is KV35, which gives $35 \times 48/8 = 210$ rpm, or 22 rad/s. The sagittal pair dominates the robot’s capability envelope. The capability analysis of Section 3.6 finds 87 % of feasible gaits limited at the cam actuator’s peak-power corner, which makes *no-load speed* rather than peak torque the axis a successor should buy. The low reduction ratios also keep reflected inertia low enough for the legs to be backdriven by hand. The teach-and-replay gait recording of Section 3.11 depends on that property.

Where the choice sits in the field. The cam and thigh requirement is a single actuator carrying more than 100 N·m of peak output torque. Filtering the database to that requirement leaves 36 commercial candidates from eight vendors. Table 3.2 ranks them on the quantity the capability analysis says is binding.

Actuator	ω_0 (rad/s)	τ_{pk} (N·m)	τ_{cont} (N·m)	Mass (kg)	Price (EUR)	$\tau_{pk}\omega_0/4m$ (W/kg)
CubeMars AKE90-8 (8:1)	22.0	170	55	1.40	424	668
RobStride RS04 (9:1)	20.9	120	40	1.42	280	442
MyActuator RMD-X12-P20 (20:1)	13.1	320	85	2.37	1003	442
MyActuator RMD-X15-P20 (72 V)	11.3	450	145	3.50	—	364
MyActuator RMD-X8-P20 (19.6:1)	16.5	120	43	1.40	593	354
MAB MA-p-100-30 (30:1)	10.1	150	50	1.10	898	343
CubeMars AK80-64 (64:1)	7.9	120	48	0.85	780	278
ZeroErr eRob110i (strain 50:1)	6.3	127	51	2.57	1270	77

Table 3.2: The ≥ 100 N·m class, sorted by mass-specific peak mechanical power. The AKE90-8 is not a compromise. Across all 130 commercial actuators in the database with complete speed, torque and mass rows, it ranks **4th** on this metric and **1st** among everything above 100 N·m. It also holds the highest no-load speed in its class and the lowest mass. At EUR 0.45 per watt of peak power it ties the cheapest entry, RobStride, and undercuts the strain-wave options by a factor of 14.

The pattern in the table is the design lesson, not the winner. Peak mechanical power per kilogram saturates around 450–670 W/kg whatever the vendor or the price. What varies is *where on the torque–speed line* a product places that power, and the reduction ratio is the knob. Strain-wave and high-ratio planetary units buy torque density and positioning accuracy at a catastrophic cost in speed. The eRob110i carries a peak torque comparable to the AKE90-8’s, costs three times as much, weighs 1.8 times as much, and delivers one twelfth the mass-specific power, because 50:1 puts its no-load speed at 6.3 rad/s. For an industrial arm that is the right trade. For a running leg it is fatal, and it is the trade most of the actuator market is built around.

The choice was correct, and correct for the stated reason. That distinction matters, because buying the same part for the wrong reason would have been easy. A selection made on *torque* density would have chosen the MyActuator RMD-X15, at 129 Nm/kg peak against the AKE90-8’s 121, and lost 49 % of joint speed. A selection made on *rated* torque density would have taken the database’s own headline ranking — the MAB MA-p-100-30 at 45.5 Nm/kg continuous, or the MyActuator EPS-RH-25-100 at 44.6 — and lost 54 % and 86 % of joint speed respectively. That headline ranking is a trap for this application, and it is the first thing the tool shows. The one metric that selects the AKE90-8 is the product $\tau\omega$, and Section 3.6 independently confirms that $\tau\omega$ is what binds.

The coupled torque–speed curve, now that R is known. An earlier draft of this section recorded that the simulation applies independent torque and speed caps but *no coupled torque–speed curve*, on the grounds that the phase resistance was unavailable. That claim was wrong. The resistance is published and the database ingested it: $R_{ll} = 0.164 \Omega$ for the AKE90-8 and 0.60Ω for the AK60-39. Both sit in the same line-to-line frame as the quoted K_t , and that pairing makes $K_m = K_t/\sqrt{R}$ exact without knowing the winding. CubeMars’ own published K_m of 0.674 reproduces $K_t/\sqrt{R_{ll}}$ to five digits, which cross-validates both numbers.

With R in hand the envelope can be stated rather than bounded. Back-EMF plus IR exhausts the 48 V bus at $K_b\omega_m = 48 - (72 \times 0.164) = 36.2$ V. That puts the corner at $36.2/0.272 = 133$ rad/s at the motor, or **16.6 rad/s at the joint**. Below the corner the actuator is current-limited and the torque cap is flat. Above it, torque falls linearly to zero at 22 rad/s. Every operating point the capability analysis actually visits lies on the flat branch with roughly a factor of two in torque to spare, the worst case being 64 N·m at 13.7 rad/s. The constant force range in simulation is

therefore not merely defensible but correct, and the omission previously flagged as “optimistic in the fast-running regime” is smaller than feared. What remains genuinely unmodelled is the fall-off between 16.6 and 22 rad/s. That fall-off is now a five-line addition to the model rather than an unmeasurable unknown.

3.2.2 What the actuator selection got wrong

Three criticisms survive the vindication above. They are given in increasing order of consequence.

The hip-roll actuator was chosen on the wrong axis, harmlessly. Its class is 45 to 130 N·m under 1.1 kg, and within that class the AK60-39 ranks 10th of 18 on mass-specific power, at 246 W/kg against 472 W/kg for the CubeMars AK10-9. It was bought for torque density, at 96 Nm/kg peak, which is genuinely near the top of the class. Hip roll is not a propulsive axis, so its low speed costs nothing measurable. The real cost lies elsewhere and was not anticipated. The 39:1 reduction gives the part $0.046 \text{ kg}\cdot\text{m}^2$ of reflected armature inertia, more than double the sagittal pair’s, plus a published 1.4 N·m of back-drive torque. The teach-and-replay workflow of Section 3.11 depends on backdriving the legs by hand, and the hip-roll axis is the one that resists. A 16:1 part, the AKH70-16 at 78 N·m, 0.88 kg and EUR 525, would have kept the torque and cut the ratio-squared inertia amplification by $(39/16)^2 \approx 6$, for EUR 132 more per side. The error is small, real, and cheap to fix.

The measured 85 % “gearbox efficiency” may not be gearbox efficiency. The database’s central methodological warning applies with force to this project’s own measurement. The datasheet K_t is tagged line-to-line. The drive reports a current over CAN whose reference frame is documented nowhere and was never established. If the drive reports q-axis current, then the correct comparator is $K_t^q = \sqrt{3/2} K_t^l = 0.333$, and the measured effective constant is 69 % of datasheet rather than 85 %. No planetary gearbox explains 69 %. If the drive reports bus current instead, the discrepancy moves the other way. A single-stage 8:1 planetary should run at 94–97 %, so 85 % was always slightly suspicious, and it was accepted only because it sat comfortably inside a plausible range. The data already recorded cannot separate the two hypotheses. The experiment that separates them is trivial: command a known current into a stalled motor and measure the DC bus current with a clamp meter. Until that is done, the peak torques of Table 3.1 carry an unquantified frame risk, on top of the geometric uncertainty Section 3.5 already acknowledges. **The correction, if it is one, is pessimistic.**

The drive was the mistake, and it was a separable purchase. The single largest sim-to-real gap in this project is the drive’s proportional-only 0.8 Hz position loop with unwritable gains (Section 3.5.3). Whose fault that is deserves precision. The database records the AKE90-8 as a bare quasi-direct-drive module, sold with no encoder and no drive board. The drive electronics were therefore an independent decision, not an inseparable consequence of choosing the best motor in the field. The database’s drive table lists at least two alternatives that were available at the time and would have removed the problem entirely. The mjbots moteus-n1 offers 100 A peak, 10–54 V, 14.6 g, EUR 137, open-source firmware and an integrated absolute encoder. The MAB MD80 v3.0 offers 80 A peak, 48 V or 60 V variants, 15 g, EUR 220, FDCAN at 8 Mbit, and position, velocity, impedance and torque modes with writable gains. Either would have cost *less* than the bundled CubeMars electronics and exposed a torque mode with a known loop. The 60 V MD80 would also have raised the bus to 60 V, for a 25 % gain on the no-load speed that Section 3.6 identifies as the binding axis.

The counter-argument is real and should not be waved away. A bare module plus a third-party drive requires mounting and calibrating an encoder, commissioning firmware, and tuning a commutation loop per joint. That is six times over, before a single line of control code runs, on a one-person thesis with a fixed end date. The integrated option was plug-and-play with a

documented CAN protocol, and it bought perhaps three weeks of schedule. Against a nine-month project that is a defensible trade.

What is *not* defensible is that the trade was made without asking the question. No vendor publishes a servo-loop bandwidth, so the figure could not have been looked up. It could have been measured on a single unit in an afternoon, with the swept-sine rig that eventually measured it a year later. And the cost of being wrong was not three weeks. It was every policy trained in the project. The database captures peak torque, rated torque, speed, mass, price, IP rating and communications. **It has no column for control-loop bandwidth or for gain writability**, because nobody knew to ask when it was built. That omission is the most useful thing this project learned about buying actuators. It is recorded here as the schema's first missing field, not as a datasheet the vendor should have written.

A flagged inconsistency for the corrected sweep. The capability analysis of Section 3.6 caps the cam actuator at $P \approx \tau_0 \omega_0 / 4 \approx 935 \text{ W}$. That expression is the maximum of a linear torque–speed line running from stall torque to no-load speed, which is the envelope of a voltage-limited brushed motor. It is not the envelope of a current-limited FOC drive. That envelope is flat at τ_{pk} up to the 16.6 rad/s corner derived above, so it peaks near $170 \times 16.6 \approx 2.8 \text{ kW}$. CubeMars' own peak-power curve for the part is annotated 1500 W . The burst figures of Table 3.10 may therefore be *pessimistic* by a factor between 1.6 and 3, which is the only correction in this entire chapter that moves in the optimistic direction. Two cautions keep it from being good news. The $55 \text{ N}\cdot\text{m}$ continuous rating sets the thermally sustained figure, that figure does not move at all, and it is the sustained figure that decides whether a record is reachable (Section 3.6.6). The 0.8 Hz drive bandwidth also caps achievable cadence far below any of these power limits. The discrepancy is recorded rather than resolved, because resolving it means re-running the sweep with a proper envelope. Section 3.6.6 lists that re-run among the outstanding items.

Cost, delivery and assembly. Four non-performance constraints bounded the search, and they are recorded here for completeness.

Cost. Six actuators at EUR 424 and EUR 393 come to EUR 2482, roughly half the hardware budget. The strain-wave alternatives in Table 3.2 would have cost EUR 7000 to EUR 12000 for strictly worse running performance, so price and performance pointed the same way for once.

Delivery. CubeMars ships to Switzerland in two to three weeks, against quote-only lead times of two to three months for Maxon's HEJ series. A slipped actuator delivery is a slipped project, which is exactly why the database records list price and stock channel.

Assembly. The integrated units bolt to a face and expose a four-wire CAN and power harness. A bare module instead needs an encoder mount, a drive enclosure and a thermal path at every joint. With 12 joints' worth of harness already in the base, that was a real consideration, and it pulled the same way as the schedule argument above.

Effort. The database itself cost about a week to build and populate across eight vendors. Such a cost is justifiable only because the result is reusable. It is published as a standalone tool, and anyone who disagrees with its verdict on this robot's actuators can reproduce that verdict.

3.2.3 Battery pack

The motor selection fixes the electrical requirement: six drives on a nominal 48 V bus, with short, violent current peaks. That is a sprint robot's duty cycle, not a marathon robot's. Usain Bolt needs under ten seconds for 100 m , and this robot would be delighted with thirty. The pack is therefore optimised for *power* density rather than for energy density.

The selection instrument. As with the actuators, the choice was made against a purpose-built database and explorer rather than against a shortlist.² It holds roughly 430 distinct cell models, 54 BMS modules and 110 commercial packs, and every row carries a source URL. It also synthesises packs from `cell + BMS + (nS, mP)`, so that “what I could build” plots on the same axes as “what I can buy”. Two of its conventions matter for what follows.

Current is a curve, not a number. The convention is the direct analogue of the actuator database’s rated, peak and momentary torque tiers. Every current rating carries a duration stamp and falls into one of three tiers. *Continuous* is the thermal steady state. *Pulse at t* is the thermal transient, interpolated as $I(t) = \sqrt{I_{\text{cont}}^2 + (I_{\text{pulse}}^2 - I_{\text{cont}}^2) t_{\text{pulse}}/t}$. *Fault* is the protection trip, which is a disconnection event and never usable power. A rating quoted without its duration fits no tier and is not a specification.

Estimated values are marked as such. Roughly half the cell rows carry at least one field inferred rather than read off a datasheet. Such a field is written **ESTIMATED<n>**, so that it still parses and can be filtered out. Every figure quoted below comes from a datasheet-sourced row.

What the database says about the pack that was built. The pack is **13S2P** of **EVE INR21700-50PL**: 48.1 V nominal, 10 Ah, 468 Wh and 1.87 kg of cells. Its BMS is rated 200 A continuous, and a software cap limits the peak to 250 A. The database corrects the specification the pack was sized against, and the correction is not cosmetic. The 50PL is a 5.0 Ah cell at **50 A continuous** and **180 A for 2 s**. This project sized the pack on a 125 A figure, which is a marketed peak with *no published duration*. That is precisely the class of number the tier convention exists to reject. The pack’s real ratings are therefore those of Table 3.3.

Tier	Cell-limited	Bus power	Binding element
Continuous	100 A	4.8 kW	the cells (not the 200 A BMS)
Pulse, 10 s	184 A	8.9 kW	the cells, then the BMS
Pulse, 2 s	360 A	17.3 kW	the 250 A software cap

Table 3.3: The 13S2P pack by current tier, from the database’s pulse model. The single figure of “250 A cell-limited” previously quoted conflates the 2-second tier with the continuous one. The pack sustains 100 A indefinitely, not 250 A.

Two consequences follow. First, the BMS is *not* the binding element at continuous, as was assumed. The cells are, at half the BMS’s rating, so the pack is over-specified on the BMS and under-specified on the cells. Second, the 250 A software cap is a 2-second-tier number enforced with no duration logic. A policy drawing 250 A for thirty seconds runs the cells at 25 C against a 10 C continuous rating, and nothing in the stack notices. That is a real safety gap, and the fix is cheap: an I^2t accumulator in the control daemon rather than a scalar clamp.

The question the sizing should have asked is whether 4.8 kW continuous suffices. Six actuators at the capability analysis’s worst observed operating point draw a few kilowatts electrical, so the pack is adequate. The margin is near 1.2 to 1.6×, however, not the 3× the 250 A figure implied.

Chemistry: right answer, wrong reasons. The original rationale ran as follows. LTO and high-rate LiPo are nominally more power-dense, but neither could be sourced as bare cells from a reputable supplier, LTO surfacing only on AliExpress and LiPo being unavailable packaged at 48 V. High-rate 21700 NMC was therefore the practical space. Sweeping the supplier landscape shows both halves of that reasoning to be wrong, and they fail in opposite directions.

²<https://github.com/nathmo/batteryReview>

LTO was rejected for a bad reason, and the rejection was correct anyway. EU-warehouse LTO specialists do exist, and the survey found them. SeLian Energy and BatteryFinds both stock Yinlong cells in Germany and Poland with published prices, and ELERIX and GWL sell into the EU from Czechia. The provenance objection therefore does not hold. What kills LTO is the number nobody checked: on the axis that matters it is *not* more power-dense. The best LTO cell in the database is the Toshiba 20 Ah SCiB at 932 W/kg and 93 Wh/kg, while the chosen NMC cell delivers 2500 W/kg and 250 Wh/kg. LTO’s reputation for power comes from its C-rate, and C-rate is a ratio. Its 2.3 V cell voltage and heavy steel construction throw the advantage away in absolute terms. A 48 V LTO pack needs 21S at 2.3 V, and at 1.24 kg per Yinlong cell that is a **26 kg** battery on a 15 kg robot. The 21S floor makes the outcome inescapable. The lightest LTO cell in the database is a 1.5 Ah 18650 at 45 g, and 21S of those carries only 76 Wh, so reaching 480 Wh at 48 V still costs 6.6 kg. No LTO configuration in the database clears the requirement under 6 kg. The chemistry was excluded on sourcing when it should have been excluded on mass. That matters, because the sourcing argument would have collapsed under thirty seconds of scrutiny while the mass argument is unanswerable.

LiPo was rejected for a bad reason too, and this one changes the answer. High-rate LiPo is sourceable as bare 1S cells, from **Swaytronic in Niederlenz**. That is a Swiss domestic supplier: no import, no customs, no VAT handling, and two-day delivery. It is the *easiest* supplier in the entire survey rather than the hardest. Its 1S cells are rated 25 C continuous and 50 C burst. Because the cells are 1S, the “no packaged 48 V module” objection dissolves, since series count is then a free parameter. Table 3.4 puts the two builds side by side.

	13S2P EVE 50PL (built)	13S1P Swaytronic 10 Ah (available, not chosen)
Energy	468 Wh	481 Wh
Cell mass	1.87 kg	2.54 kg
Continuous current	100 A (4.8 kW)	250 A (12.0 kW)
Pulse, 10 s	184 A (8.9 kW)	500 A (24.1 kW)
Cell cost	EUR 96	EUR 559
Cells to assemble	26	13

Table 3.4: The built pack against the high-rate LiPo pack the survey shows was available from a Swiss domestic supplier. At the sprint timescale the project explicitly designs for, the alternative delivers $2.7\times$ the current for 0.67 kg and EUR 463.

The comparison is unflattering on the criterion the section itself declares. The stated design target is a sprint, so the ten-second tier is the tier that should have driven the choice. On that tier the LiPo pack delivers 2.7 times the current. The mass penalty is 0.67 kg carried in the *torso*, where Table 3.11 prices it at -0.11 (m/s)/kg, or about -0.07 m/s. Against an actuator budget of EUR 2482, the extra EUR 463 is real but not decisive. On the project’s own stated optimisation criterion, the built pack was the wrong pick.

Three honest counterweights exist, and none of them was the actual reason at the time.

Cycle life. High-rate LiPo runs a few hundred cycles against 500 to 1000 for a 21700, and this robot is a lab platform that will see thousands of trials.

Assembly. Twenty-six cylindrical cells spot-weld into a rigid, self-supporting brick that carries load in the base. Thirteen pouch cells instead need a compression fixture, a case, and tabs joined without cooking them. That is more work, and it fails worse.

Lab safety. High-rate LiPo attracts storage and charging restrictions that a cylindrical NMC

pack behind a certified BMS does not. A battery that cannot be left charging overnight in the lab carries a schedule cost that no datasheet shows.

Taken together, the three are a genuine defence of the built pack. They are not the defence that was recorded. “We chose NMC because LiPo was unsourceable” and “we chose NMC because pouch cells are a fire-safety and assembly liability we did not want on a shared bench” differ by the difference between an error and a trade-off.

Build versus buy. Building was necessary, and the database establishes that rather than assuming it. Of 110 commercial packs surveyed, the 48 V space holds almost only two things. The first is stationary storage: 16S LFP rack and wall units from 5 to 32 kWh, weighing 29 to 370 kg, useless here. The second is e-bike packs. The closest commercial match to the requirement is a Swaytronic 48 V 13S3P 10.35 Ah at 497 Wh, 2.2 kg and EUR 485. That is essentially the pack this project built, at twice the cost and with no assembly labour. It fails on exactly one axis: as an e-bike pack it is rated in the 40 A class, roughly $2.5\times$ short of the requirement. **Nothing in the commercial 48 V market is built for a sprint duty cycle.** That finding is what justifies building a pack at all, and it took a 110-row survey to state rather than assume.

Cost, delivery and assembly. Four constraints again bounded the choice.

Cost. EUR 96 of cells, plus roughly EUR 90 to EUR 165 for a 13S 200 A smart BMS (JBD and JKBMS both list suitable units), plus nickel strip and holders. Call it EUR 250 against EUR 485 for the nearest commercial pack. That is a saving, but it was not the reason.

Delivery. The surveyed supplier space is stratified by border, not by price. A Swiss supplier ships in two days. EU sellers ship in under a week, with Swiss import VAT at the border. China-direct suppliers ship from EU warehouses in one to three days, but air freight frequently bars loose cells. US shops offer the best transparency and the lowest minimum order quantity of anyone, but they cannot legally air-freight loose lithium to Europe, and were excluded on that basis alone.

Minimum order quantity. Single cells are available for 18650 and 21700 NMC. LTO ships in six-cell matched sets and large-format LFP in four-packs. The minimum order quantity, not the price, is what actually gates a prototyper out of a chemistry.

Assembly. Twenty-six spot welds of nickel strip take an afternoon with a lab welder, and yield a structural element. That constraint favoured cylindrical cells most quietly of all, and it was never written down as a criterion.

Voltage sag is probably the largest unmeasured quantity on the robot. One consequence of the *drive* selection became visible only once the CAN traffic was sniffed (Section 3.5.7). The drives publish position, speed, current, temperature and an error flag, but **no bus voltage**. Pack sag under a sprint is therefore not observable from the drives at all. Observing it would need external sensing on the Pi or on the BMS. Until such sensing exists, the simulation models voltage droop as a randomised within-episode torque derating (Section 3.8.12) rather than as a measured quantity.

The database’s internal-resistance model shows that sag is not a second-order effect, and connects it to the actuator argument above. A pack’s sag ceiling is $(V_{\text{nom}} - V_{\text{cutoff}})/R$, and a 13S2P arrangement multiplies cell resistance by $13/2 = 6.5$. The 50PL row carries *no* datasheet internal resistance, and the database flags that gap rather than filling it. A high-drain 21700 sits around 12 to 15 m Ω , which puts the pack near 80 to 100 m Ω . At the 100 A continuous tier that is 8 to 10 V of sag. At the 184 A ten-second tier it is 15 to 18 V, which takes a nominally 48 V bus down toward 30 V.

If that estimate is even approximately right, it is the most important unmeasured number on the robot. Section 3.6 establishes that no-load speed is the binding axis, and no-load speed is *linear in bus voltage*. A 30 % sag is therefore a 30 % loss of joint speed, arriving exactly when the policy asks for speed, on a robot that cannot see it happen. Sag also sharpens the LiPo comparison of Table 3.4 in the same direction. A 13S1P pack multiplies cell resistance by 13 rather than by 6.5, but its cells carry roughly an order of magnitude less resistance each, so its sag would be a fraction of the figures above.

The figures in this paragraph are an inference from a typical cell resistance, not a measurement, and they are presented as such. The number is not the point. The point is that the cheapest remaining experiment in the project addresses a plant parameter with more leverage on top speed than most of what Section 3.5 did measure. That experiment is a voltmeter across the pack during a sprint, or a single datasheet lookup for the 50PL’s DC internal resistance. That it went unmeasured is the failure mode this chapter documents everywhere else: the quantity nobody instrumented is the quantity that moved.

3.2.4 Compute

Most of the laboratory’s exoskeletons run on a BeagleBone Black. The board supports CAN natively and runs Linux, but its SSH connection was too slow to work with, so it was rejected. Its main selling point is the real-time coprocessor, a substantial microcontroller that runs alongside the main CPU for time-critical tasks. That coprocessor needs custom firmware, and its performance benefit for this application is still to be demonstrated.

A Raspberry Pi already in the laboratory was used instead, because I had far more experience with that board. The choice required buying a CAN hat that talks over SPI. A second hat carrying an accelerometer, the Waveshare Sense HAT (B), was bought for ease of integration.

The board is a **Raspberry Pi 3 Model B Rev 1.2**: four Cortex-A53 cores, aarch64, kernel 6.12, running a stock non-realtime Linux. An mcp251xfd controller over SPI provides CAN, configured as **two independent 1 Mbit buses**. Bus `can0` serves the right leg and `can1` serves the left, with three motors each: CAN id 104 for hip roll, 105 for cam and 106 for thigh. The arbitration id encodes both node and command, as `id | (command << 8)`.

The Pi also hosts a WiFi access point and a web interface, which drives the robot during bring-up and removes the need for a tethered console. That interface is not only a convenience. It is the instrument through which most of the measurements in Section 3.5 were taken.

This section originally left the loop-rate and jitter question open, and Section 3.5.6 settles it by measurement. The Pi sustains a closed 200 Hz control loop at half its time budget, and 200 Hz is the correct deployment rate.

3.2.5 Sensors

Beyond the motor encoders, the only proprioceptive sensors are the base IMU and the motor current estimates. Contact sensors were considered, because the literature suggests that impact timing and direction improve RL convergence, but they were never fitted.

The IMU is an **ICM-20948** on a Waveshare Sense HAT (B), bolted underneath the base and read over I2C. It is configured at $\pm 4g$ and $\pm 1000dps$, with the internal low-pass at a 50 Hz bandwidth, and it is sampled at 200 Hz. Its measured properties are summarised below. The full characterisation, with the provenance of every figure, is in the project repository.

	Accelerometer	Gyroscope	Source
Resolution (1 LSB)	0.1221 mg	0.0305 dps	measured
Noise (RMS at rest)	1.8 mg	0.08 dps	measured at rest
Noise density	$218 \mu\text{g}/\sqrt{\text{Hz}}$	$0.0096 \text{ dps}/\sqrt{\text{Hz}}$	at or better than datasheet
Turn-on bias	absorbed by mount cal.	$[1.03, 0.11, -0.53] \text{ dps}$	measured, repeatable
In-run bias stability	—	0.010–0.012 dps	measured

Table 3.5: Measured IMU noise characteristics. Quantisation sits roughly $15\times$ below the noise floor on both channels, so widening the ranges to survive footfall impacts costs nothing in effective resolution.

Two properties matter more for control than the noise figures do.

Latency. The path from a physical event to a published value takes 10 to 20 ms. It comprises the DLPF group delay (5 to 10 ms, from the datasheet, not verified on this part), output-rate quantisation (0 to 4.3 ms), the I2C read (1.7 ms) and the poll period (0 to 5 ms). Against a 5 ms control period, that is two to four whole control steps. The effect is larger than the sensor noise, and the simulation models it as such.

The attitude filter dominates, not the sensor. A 6-axis Madgwick filter with $\beta = 0.06 \text{ rad/s}$ runs at the poll rate. Its accelerometer correction is rate-limited to roughly $3.4^\circ/\text{s}$ whatever the size of the error. The published attitude is therefore overwhelmingly gyro-driven, and the accelerometer acts as a slow trim. Roll and pitch are gravity-referenced and absolute. Yaw is pure gyro integration and drifts, so the gyro zero must be captured on a stationary robot each session. A policy that consumes fused attitude rather than raw rates forces the simulation to model this filter, not merely the sensor.

The magnetometer is deliberately not fused. It sits centimetres from two brushless motors and the battery, and its reading moves when they do.

A methodological trap worth recording. Noise figures taken while the robot swayed on its test rig measure the sway, and nothing in the resulting number says so. That mistake produced a yaw-axis noise figure roughly $8\times$ too high. It also produced a filter sweep in which a *narrower* low-pass appeared to measure *more* noise. Both results looked like plausible sensor data. The benchmark tool now gates every noise figure on a peak-to-peak motion check. Every published figure also carries a provenance label: measured at rest, motion-independent, datasheet-unverified, or uncharacterised. The uncharacterised list is published rather than guessed at. It holds the temperature dependence of the gyro bias, vibration rectification under footfall impact, accelerometer cross-axis sensitivity, and clipping behaviour at $\pm 4 \text{ g}$ during real impacts.

Getting the rate the sensor was configured for. The IMU loop initially reached only 190 to 194 Hz against a nominal 200 Hz. The missing ticks were not the IMU's. The air, pressure, light and ADC channels were being read inline on the same thread, and those drivers spend nearly all their time asleep waiting for a conversion. The humidity sensor alone sleeps 15 ms, which is three whole control ticks. Moving the slow chips to a second thread, with a lock serialising only the bus transfers, brought the IMU to 199.5 Hz. As a side effect it reduced the CAN daemon's late ticks from 11.5 % to 9.8 %. The remaining slips are operating-system scheduling.

3.3 Robot modelling

The robot was re-assembled in Autodesk Inventor, chosen for fluency. Each limb was exported as STEP with its origin centred on its joint, then assembled in Fusion 360 and exported to URDF with a plugin. The path was not smooth. The detours are worth recording, because they are the normal experience of CAD-to-simulation work.

- **Inertia matrices from the mechanical engineer’s SolidWorks model arrived in the wrong orientation**, and after repeated attempts the SolidWorks URDF export never yielded a usable file. The working pipeline therefore validates every CAD-supplied inertia tensor against its reference frame explicitly, rather than trusting the export.
- **URDF cannot express this robot at all** (Section 3.4). The parallel knee closes a kinematic loop, so a URDF is at best an open-tree approximation with a floating pushrod. The actual model is MJCF with an explicit `<equality><connect>`. Both files are exported and kept, and only the MJCF is simulated.
- **Validation is a tool, not an inspection.** A dedicated tool discovers plausible inertia matrices from the physical robot, by moving the legs in the air with the base fixed, inside the calibrated safe workspace, then compares them against the CAD values. Section 3.5.5 reports the full dynamic identification this grew into. Its bottom line for the model has two halves. The CAD centres of mass *survive* scrutiny, because a free 18-parameter fit explains at most 11 to 19 % of the observed gravity residual, so the CAD values were kept. The link inertia *tensors* remain CAD placeholders scaled by measured mass ratios, which is why the domain-randomisation range on inertia stays at $\pm 25\%$ while the mass ranges narrowed.
- **The kinematic model is consistent with the measured robot.** The MJCF closes the 4-bar at 100 % of the backdriven-workspace poses recorded on the real robot: 1383 of 1383 on the left, 1406 of 1406 on the right, at a residual of 10^{-14} . The test is one-sided, since it would catch only a model band *smaller* than reality, but it is reassuring.
- **The exported model is not left/right symmetric, and symmetrising it was tried and reverted.** An audit found 17 of 30 mirror-compared quantities off, including a 1.2° rotation in one hip frame and sub-millimetre centre-of-mass offsets. The near-singular 4-bar amplifies those into 6.5° of hip-roll difference in the standing keyframe. A tool that mirrors the model exactly was built, reducing the worst error from 1.2° to 2×10^{-5} , and it was *not* applied. Three findings stopped it. The crooked stance survives symmetrisation, because the gravity settle bifurcates into a lean of about 7° even from a symmetric start, so the asymmetry only chose *which way*. An unexplained regression in the pitch-reflex kick-arrest test failed the smoke gate. And the deployment lineage had already chosen to *randomise* the asymmetry rather than deny it (Section 3.8.12). The tool is kept. The premise that the asymmetry was the bug was measured to be wrong.

Mass. Every segment was weighed rather than taken from CAD:

Segment	Mass	Count
Base (torso, electronics, battery)	5764 g	1
Hip (incl. abduction and sagittal motors)	3271 g	2
Cam	66 g	2
Thigh	483 g	2
Push rod	71 g	2
Ankle spring assembly	249 g	2
Shin	324 g	2
Foot	222 g	2
Total	15 136 g	

Table 3.6: Weighed segment masses. These supersede the CAD placeholders, which totalled 12 828 g.

The correction is neither small nor uniform. The robot is **18 % heavier** than the CAD model every policy up to that point had trained against. The error also concentrates in exactly the worst place. The simulation’s shin body — shin plus ankle spring assembly, weighed at 573 g — was carrying a 222 g placeholder, a factor of 2.6 too light. Table 3.11 shows why that matters more than the total does: distal leg mass costs roughly nine times what torso mass costs in top speed. Section 3.5 discusses the consequences.

3.4 Simulation

Simulation runs in MuJoCo. The robot’s topology forced that choice more than preference did.

3.4.1 URDF cannot describe this robot

The parallel knee closes a kinematic loop. The cam actuator drives a pushrod whose distal end ties to the shin below the knee, so hip, cam, pushrod, shin and thigh form a planar 4-bar. URDF is a *tree* format and cannot express that closure. The exported URDF therefore describes a different robot, one with a floating pushrod. Any simulator that consumes it directly produces dynamics that are plausible and wrong.

MuJoCo’s MJCF closes the loop with an explicit `<equality><connect>` constraint. Both an open-tree URDF and a closed MJCF are exported and kept, and only the MJCF is used for physics. The practical consequence for control is significant, and Section 3.8.3 discusses it further. Cam and thigh are *not* independent. The loop assembles over only a diagonal band of the nominal joint box, and a target commanded outside that band is not merely hard to track but physically impossible.

3.4.2 Compute backend

Physics runs on the CPU throughout, because this stack has no GPU physics backend. Training nonetheless runs measurably faster with the policy network on a GPU: 2900 environment-steps per second against 1530 on the same cluster node, a factor of 1.9. Matrix throughput is not the cause. The network is a two-layer MLP whose forward pass costs 117 μ s even on the robot’s Raspberry Pi. The cause is core contention. PyTorch on the CPU steals cores from the parallel MuJoCo workers, and moving the network off the CPU returns those cores to physics.

3.4.3 How do you know a simulation result is real?

The reward function demanded a particular discipline, described in Section 3.8.2, where the policy exploited every loophole it was offered. The *physics* turns out to demand the same discipline. A simulation bug does not usually announce itself. It produces a clean, converged, entirely plausible learning curve for a robot that does not exist. Four examples follow. All four were found while setting up a single ankle study, and each would have silently produced a wrong answer.

- **Joint limits are not rigid by default.** MuJoCo’s default limit softness let body weight push 3.4° past the stop of a supposedly rigid strut. That is a rubber bar, not a rigid one, and it would have quietly given the study’s rigid arm a compliant ankle. Stiffening the constraint reduced the penetration to 0.08° .
- **A constraint that ships inactive is inactive.** The rigid arm re-settled its stance with a *free* ankle, because the locking equalities switch on only at episode reset. The arm was being initialised in a pose its episodes never visit, 268 mm lower than intended.
- **A safety criterion measured in the wrong frame.** The workspace termination measures the toe in the *base* frame, so it cannot distinguish a collapsed ankle from a folded 4-bar. One arm stood outside the threshold from its own reset pose, and every episode terminated 0.1 s in. The criterion had to be re-referenced, per arm, to that arm’s own settled stance.
- **A stale keyframe.** The shipped stance is an equilibrium of one particular ankle law. Arms with a softer ankle start away from equilibrium and lurch at every reset.

A fifth class is environmental rather than physical. The local development machine ran MuJoCo 3.7.0 and the cluster ran 3.3.7. A weld constraint that held on the newer solver *gave way* on the older one, drifting 0.86 rad over a 60-step rollout. The “rigid” arm was silently compliant on the cluster alone. Anything solver-marginal must therefore be tuned against the older version.

The countermeasures are structural rather than heroic. A smoke-test suite builds and briefly runs all 123 training presets before any long launch. Self-tests inside the analysis tools check analytic Jacobians against finite differences, and loop closure against the model’s own keyframe. Assertions fail loudly when a plant does not match what its preset claims it is.

3.5 Plant identification: replacing assumptions with measurements

Everything described so far, and every training result in Section 3.8, was obtained against a model whose parameters came from CAD and from datasheets. Between 2026-08-04 and 2026-08-06 those parameters were measured on the assembled robot. This section reports what changed.

Table 3.7 is the summary, and it is the most important table in this chapter. No individual row is the reason. **Every correction points the same way.** The real robot is heavier, weaker, slower and softer than the model it trained against. That is a finding about how simulation-first development fails, not an apology for it.

Quantity	Assumed	Measured	Consequence
Total mass	12.83 kg (CAD)	15.136 kg weighed; shin body 2.6×	every policy to date trained on a robot that does not exist
Peak joint torque	170 N·m (datasheet)	144.5 N·m ($K_t \approx 85\%$)	simulation $\approx 18\%$ stronger than hardware
Drive position loop	≈ 13 Hz servo	0.8 Hz + 25 ms delay	30 % of commanded amplitude, 94° late, at 2.5 Hz
Ankle spring	$k = 28.65, 12.8$ N·m preload	$k = 41.4$, no preload	145 mm of sag; fails statics at 0.5 BW
Control loop	200 Hz, ms-scale jitter	200.0 Hz, 0.14 ms sd, 50 % of budget	200 Hz confirmed; 50 Hz is measurably worse
CAN load	assumed near the limit	26 % of one 1 Mbit bus	the bus was never the constraint
Ground friction	$\mu \approx 1$	40° slip angle, $\mu = 0.84$	train deliberately at 0.35–0.60

Table 3.7: Assumed versus measured plant parameters. Every row moved in the pessimistic direction.

3.5.1 Mass

The weighed masses of Table 3.6 were baked into the simulation model, replacing the CAD placeholders. The inertia tensors were scaled by the per-body mass ratio, and the stance keyframe was re-settled under the new load.

The decision worth recording is that the correction was *not* made an experimental variable. An earlier arrangement kept a CAD plant and a measured plant as two selectable model files. Every preset that had not explicitly opted in therefore kept training on the 12.83 kg robot, and that was all of them but two. Splitting the plant in two had made the wrong one the default. There is one robot, it weighs 15.14 kg, and the model file says so.

The accepted cost is that warm-starting a new run from a pre-correction checkpoint is now a cross-plant transfer. Those weights, and their observation normalisation statistics, were fitted to a robot 18 % lighter, with a shin 2.6 times too light.

3.5.2 Peak torque is what the gearbox delivers, not what the datasheet quotes

The actuator force ranges in the model carried the datasheet peak torque, which is the torque at an ideal torque constant. The gearbox does not deliver it.

K_t was measured by clamping a known 1.552 kg mass to the shin and repeating a quasi-static sweep with the mass and without it. The method matters. Fitting the two runs *together* with an indicator column makes the mass term’s coefficient exactly $1/K_t$, because a known mass on a known kinematic lever arm involves no link centre of mass anywhere. That breaks a degeneracy which had defeated the identification until then. Otherwise K_t and the gravity-model gain are perfectly confounded, and only their product is observable.

The effective joint K_t came out at 82 to 89 % of datasheet, consistent with a gearbox at roughly 85 % efficiency. Peak torque scales with K_t at the same peak current, so the model was updated. The measured cam and thigh actuators moved from 170 to 144.5 N·m. The hip-roll actuator moved from 72 to 61.2 N·m. The second figure is the same efficiency *transferred*, not measured, because no datasheet K_t is published for that part to fit against. It is probably optimistic,

because a 39:1 reduction needs more gear stages than an 8:1.

The remaining uncertainty is geometric rather than physical. The mass was clamped by hand, K_t trades against its position at about 1.9 % per centimetre, and the perpendicular standoff of the clamp is known only to 5 to 10 cm. Across that box K_t spans 82 to 98 % of datasheet. Settling the figure needs the mass on the shin centreline at a measured distance, not more analysis of the existing data.

3.5.3 The drive’s position loop is 0.8 Hz

The drive’s position loop is the single largest modelling error found in the project. It stayed invisible until a swept-sine excitation was run on the assembled robot.

Frequency (Hz)	0.8	1.1	1.4	2.0	2.5	2.9
Gain	0.71	0.60	0.53	0.38	0.30	0.28
Phase (deg)	−48	−61	−71	−86	−94	−104

Table 3.8: Measured closed-loop response of the drive’s internal position loop, identical on both legs to two decimals. A gain of 0.71 at -48° is a first-order corner, so a single pole fits it: 0.8 Hz, plus about 25 ms of transport delay.

The simulation models these actuators as position servos with $k_p = 200$ and $k_v = 5$, which is critically damped at roughly 13 Hz. The real drive rolls off at 0.8 Hz. That is a bandwidth mismatch of more than an order of magnitude, against a learnable gait frequency range whose top is 4 Hz. At 2.5 Hz the hardware executes 30 % of the commanded amplitude, 94° late. **Every policy in Table 3.14 assumes authority the hardware cannot deliver.**

Three consequences follow.

First, the unexecuted part of a command *is* the tracking error, and it tends to the full commanded amplitude as frequency rises. Any high-frequency content therefore caps the usable excitation amplitude near 15° , whatever the workspace allows. That cap is why an early identification sweep tripped the safety limit six seconds in, on the joint with the most room. Excitation amplitude now solves against travel, no-load speed and tracking error together.

Second, **inertia cannot be excited in position mode at all**. Obtaining real joint accelerations needs either negligible amplitude at high frequency, or current-mode excitation. The quasi-static runs are the valuable data, and always were.

Third, and most consequential for the project, the fix is architectural rather than a gain change. The drive’s cascaded position loop is proportional-only, and nothing in the runtime can write its gains. The measurement points to current-mode commands with the position PD closed on the Pi at 200 Hz. It also points to a torque-source actuator in simulation, with a proper torque–speed envelope, in place of a fictional stiff servo. Section 3.8.1 discusses why that partially reverses a decision taken at the start of the project.

An existing policy cannot be adapted to the measured drive simply by switching the drive model on. Applying 0.8 Hz in one step to a converged walking policy collapsed it, with nothing else changed: episode length fell from 1448 to 251, and return from +511 to -78 . The drive model is therefore ramped in as a curriculum, logarithmically in frequency. A linear-in-Hz ramp would do almost nothing for most of its length and then fall off a cliff, because the equivalent filter coefficient is strongly nonlinear in bandwidth.

3.5.4 The ankle spring

The fabricated spring measures $k = 41.4 \text{ N}\cdot\text{m}/\text{rad}$ with **no preload**. The model assumed $k = 28.65$ with $12.8 \text{ N}\cdot\text{m}$ of preload at stance. Section 3.8.5 treats the consequences, because they overturn a conclusion drawn from the reinforcement-learning experiments.

3.5.5 System identification: including what it could not resolve

Beyond the individual parameters above, a full dynamic-parameter identification was attempted. Its limits are as informative as its results.

Method. Inverse dynamics through the closed loop was rejected. The 4-bar is near-singular over part of the operating region, and the resulting inverse blows up. Identification is instead a per-joint linear regression of measured current against four terms: a gravity-torque model, viscous and Coulomb friction, an inertia term and an intercept. A constrained Jacobian differentiates the loop closure out.

Four estimator bugs, each of which produced a confident wrong answer. A missing sign convention on the current column made every K_t come out negative. The joint mapping never mirrored the right leg, which put the reconstructed configuration outside the assemblable band and left a loop residual of 0.78 m instead of 10^{-8} . That residual reproduced identically on three separate recordings, so it was clearly not a bad capture. Two further faults each flipped the sign of K_t independently: pooling rows from joints a given excitation run did not move, and omitting the intercept. Both flip the sign because the drive's DC current offset then lands in the gravity column. The fourth bug was over-regularisation of the loop Jacobian. The problem is well conditioned in this pose region, and the damping alone inflated the fitted gravity gain by 20% . With the first two bugs fixed, the two legs agree to 5 or 6% rather than disagreeing in sign, and that cross-validation is what says the fixes are right.

The rejected result. With K_t pinned by the known-mass test, the model still under-predicts gravity torque by a factor of 1.3 to 1.75 . The obvious explanation is that the CAD centre-of-mass offsets are wrong. That hypothesis was tested and **rejected**. Gravity torque is linear in each body's centre of mass, so the test is a single linear solve. With all 18 parameters free, no regularisation and no plausibility constraint, moving mass explains at most 11 to 19% of the residual. Ten of the eighteen directions are pure null space, and that null space is where physically absurd fitted offsets come from. The CAD centres of mass therefore stand. Per-body mass, load-dependent stiction and base tilt were tested and rejected on the same grounds. One decisive test ruled out friction of any origin. Friction opposes motion and therefore reverses sign with the direction of travel. Fitting the gravity gain separately on up-strokes and down-strokes puts both on the same side of unity, for all six joints. The residual is conservative, and no friction model can produce it.

What remains lies outside the span of every rigid-body inertial parameter. That points at the kinematic model rather than at the mass distribution, and it has not been resolved.

Why this is worth reporting. An identification that returns a number is easy to over-trust. This one returns two things. The first is a measured, repeatable, per-joint gravity gain, which can be applied as a correction or as a randomisation range without being understood. The second is an explicit statement that the mass distribution is not the explanation. Knowing which parameters the experiment *could not* resolve, and why, is what stops the next person fitting the same null space again.

3.5.6 Loop timing and CAN

The whole control loop was benchmarked on the robot. One tick drains both CAN buses, runs the policy forward, and writes six commands. The measurements were taken with the web interface running, because that is the realistic case: the interface consumes roughly 70 % of a core.

Rate	Mean period	Jitter (sd)	p99	Max	Budget used
50 Hz	20.148 ms	2.151 ms	22.41 ms	49.29 ms	23 %
100 Hz	10.000 ms	0.251 ms	10.40 ms	12.28 ms	37 %
200 Hz	5.000 ms	0.141 ms	5.21 ms	7.46 ms	50 %
400 Hz	2.504 ms	0.193 ms	3.24 ms	5.13 ms	73 %

Table 3.9: Measured closed-loop timing on the robot’s Raspberry Pi 3B, with the web interface competing for CPU.

Three results follow, and all three contradict assumptions made earlier in the project.

The drives broadcast their state at exactly 200.0 Hz, unprompted. All six motors do so, on both buses, at a period of 5.00 ms and a p99 of 5.03 ms. The broadcast is free-running and continues with the control software stopped, so it is not poll-triggered. The sensing ceiling is therefore exactly 200 Hz, and it matches the control stack. A 50 Hz loop would discard three of every four sensor frames.

200 Hz is clean, and 50 Hz is reproducibly worse. The 50 Hz row above is not frame loss, because the interface counters show nothing dropped. It is a scheduling artefact of the longer sleep, combined with a drain three times more expensive because four times as many frames have accumulated. The case for dropping to 50 Hz had been compute headroom, and compute is not the constraint. Policy inference costs 117 μ s on average and 144 μ s at the 99th percentile, which is 2.3 % of a 5 ms budget.

The CAN bus was never near its limit. Six motors, counting command plus reply, occupy about 26 % of one 1 Mbit bus at 200 Hz, spread across two buses.

The conclusion is to deploy at 200 Hz, and no 50 Hz fallback is worth having. None of this helps with the real bottleneck, and that deserves emphasis. Running the outer loop at 200 Hz cannot compensate for an actuator that executes a quarter of the command 94° late (Section 3.5.3). What the measurement does establish is that the Pi holds ample budget to close the position loop itself, which is the proposed fix.

3.5.7 The CAN protocol, as measured

The protocol was sniffed on both live buses rather than read from documentation, and the difference mattered. Exactly two frame types exist per motor. Outbound is the command frame. Inbound is a single feedback frame at command byte 0x29, at exactly 200 Hz, using all eight bytes for position, speed, current, temperature and an error flag.

The VESC-style extended status frames that would report input voltage and duty cycle are absent. One hypothesis held that these drives are a VESC derivative with the extra status packets available, and that hypothesis is measurably false. The command byte is 0x29 rather than VESC’s 9, so the vendor renumbered rather than inheriting. Two consequences follow. The electrical route to K_t is closed, which is why the known-mass test of Section 3.5 was necessary. And pack sag cannot be observed from the drives.

One process note is worth recording. Nothing in the codebase decoded the command byte, because every consumer masked it off and keyed on the node id. A comment asserting the wrong command number therefore sat unnoticed in dead code for months. An assumption that is never exercised is never falsified.

3.6 Achievable gait: actuator and mass-distribution limits

The robot’s maximum running speed was estimated independently of reinforcement learning, by a first-principles analysis of the mechanical system. RL demonstrates a lower bound on performance. This analysis provides the upper bound that the robot’s kinematics, dynamics and actuator capabilities impose.

3.6.1 Method

The analysis runs directly on the MuJoCo model (`training/model/dash01.xml`), which keeps it consistent with the model used for policy training. Candidate gaits are parameterised by duty factor D , contact length L_c , ride height h and cadence.

For each candidate gait, a simplified running model gives the stride length, flight time, swing duration and ground reaction forces. The leg Jacobian then gives joint torques and velocities throughout the gait cycle.

A gait counts as feasible only if every joint stays inside the admissible 4-bar assembly region while respecting the actuator torque, speed and power limits. The parameter space was explored exhaustively to find the maximum feasible running speed.

The implementation was validated against MuJoCo. Static equilibrium predictions agree within 1 %, and the analytic Jacobian matches `mj_jacGeom` to better than 10^{-10} .

3.6.2 Results

Regime	Top speed	Limiting constraint
Continuous operation	1.8 m/s	Continuous actuator torque (55 N·m)
Cadence ≤ 6 steps/s	3.7 m/s	Cam actuator peak power
Cadence ≤ 8 steps/s	4.4 m/s	Cam actuator peak power
Unconstrained optimum	6.9 m/s	Cam actuator peak power

Table 3.10: Maximum running speed predicted from the robot model assuming a 350 N·m/rad ankle stiffness. Only the first regime is thermally sustainable; higher speeds correspond to burst operation. **Plant epoch: CAD masses (12.83 kg), datasheet peak torque (170 N·m).** Both are now known to be wrong; see Section 3.6.6.

At the practical optimum of six steps per second or fewer, the predicted gait has a duty factor near 0.45, a stride length of 1.1 to 1.3 m, and a contact length of 0.25 to 0.30 m. Its peak vertical ground reaction force is about 3.5 body weights, or 440 N, and its ground contacts last between 36 and 81 ms.

The predicted continuous-speed limit of 1.8 m/s is already lower than the 2.6 m/s the RL policy achieved, which shows that the learned controller relies on burst actuator capability. Motor temperature must therefore be validated experimentally before any sustained hardware operation. On the corrected plant of Section 3.6.6 the margin disappears entirely.

The unconstrained optimum falls at 12.4 steps/s. That is the same high-cadence solution reinforcement learning discovered repeatedly. The agreement confirms that the high-cadence behaviour originates in the system dynamics rather than in a simulator artefact.

3.6.3 Limiting factors

Swing-leg actuator power limits maximum speed, not kinematic reach.

The dominant constraint is the cam actuator during swing. It demands about 878 W, namely 64 N·m at 13.7 rad/s, against a theoretical maximum output of 935 W. Raising actuator no-load speed would therefore improve achievable running speed more than raising peak torque would.

Stance loading is the second limiting factor. Peak ground reaction forces demand about 167 N·m of the available 170 N·m, which leaves minimal torque margin.

Kinematic reach does not limit the result. The leg spans a theoretical fore-aft workspace of 1.06 m, but only 0.20 to 0.35 m of that span is usable in practice. Near the edge of the assembly region the 4-bar's transmission ratio deteriorates. Joint velocities then rise rapidly, and achievable running speed falls.

Introducing a flight phase improves performance substantially. Reducing the duty factor from 0.85 to 0.45 raises the predicted speed from 4.6 to 6.9 m/s, and demands only a 1.2 cm vertical excursion of the centre of mass. Lower duty factors demand ground reaction forces above 4.5 body weights, and are therefore infeasible.

3.6.4 Sensitivity to mass distribution

Added mass location	dv/dm
Uniform scaling	−0.14 (m/s)/kg
Torso payload	−0.11 (m/s)/kg
Entire leg	−0.43 (m/s)/kg
Distal leg only	−0.93 (m/s)/kg

Table 3.11: Sensitivity of predicted maximum speed to additional mass.

Running speed is considerably more sensitive to distal leg mass than to torso mass. Mass added at the feet, the shin or the distal linkage costs nearly an order of magnitude more than the same mass added as a torso payload.

On the current design, removing 0.22 kg from each distal leg raises predicted top speed by about 0.5 m/s. Adding 5 kg to the torso produces a comparable reduction. The result validates the heuristic that guided the leg design, namely that distal inertia should be minimised.

3.6.5 Ankle stiffness

A static load analysis independently reproduced the ankle stiffness threshold that the reinforcement-learning experiments had already shown.

The passive ankle spring, at 28.65 N m/rad, supports only about half a body weight before it collapses under positive mechanical feedback. The actuators meanwhile operate well below their torque limits. Raising the stiffness progressively improves load capacity, and about 350 N m/rad is needed to support two body weights and hold a stable stance.

The analytic load model and the reinforcement-learning experiments agree, which validates the stiffness threshold independently. A separate static end-effector force map (Section 3.11.1) reproduces the same conclusion from a third direction.

3.6.6 The corrected ceiling

The figures above were computed at 12.83 kg with a 170 N·m peak torque. Section 3.5 shows both to be wrong, and wrong in the direction that hurts most:

- Peak torque falls from 170 to 144.5 N·m. Since 87 % of feasible gaits are limited at the cam actuator’s peak-power corner, this is a direct hit on the binding constraint.
- Distal leg mass rises by roughly 0.70 kg across both legs (the shin body from 222 to 573 g), at a sensitivity of -0.93 (m/s)/kg from Table 3.11.

Propagating both corrections through the sensitivity figures gives roughly **2.7 m/s credible burst and 1.3 m/s thermally sustained**, against 3.7 and 1.8 m/s on the CAD plant. That changes the headline comparison of the whole project. The learned policy’s 2.6 m/s no longer sits comfortably inside the envelope. It sits essentially *at* the burst ceiling, and at roughly twice the sustained figure. For context, the bipedal 100 m record averages about 4 m/s, and a 100 m dash is about 25 s of continuous running against a continuous torque rating. The *sustained* number is therefore the one that decides whether a record is reachable.

The honest conclusion is that this chassis is a walking and jogging demonstrator with an unusually good measurement pedigree, not a record contender. The design implications for a successor fall directly out of the analysis above. Bus voltage comes first, because speed scales linearly with no-load speed and not at all with peak torque. Distal mass comes second.

These corrected figures propagate the sensitivity coefficients. They are not a re-run of the full feasibility sweep. The sweep should be repeated properly on the corrected plant before submission, and the tooling to do so now exists in validated form.

3.6.7 Limitations

The analysis provides an optimistic upper bound on performance. It neglects torso attitude stabilisation, lateral dynamics, impact losses, control bandwidth limits and the coupled actuator torque–speed envelope. The last of those is no longer a small omission. Section 3.5.3 measures the drive’s position loop at 0.8 Hz, so the assumption that a commanded joint trajectory is actually executed fails well below the cadences this analysis assumes.

Inertia tensors also remain approximate. The segment *masses* are now weighed (Table 3.6), and the model’s tensors were rescaled by the per-body mass ratio. The tensor *shapes* are still CAD, and the identification of Section 3.5.5 could not resolve the mass distribution any further. Table 3.11 shows the result is most sensitive to exactly that quantity, which makes it the largest remaining uncertainty in the analysis.

3.7 Toy problem: one joint, end to end

Committing a 12-DOF robot to an unproven pipeline is expensive. The complete chain — simulate, train, export, deploy on embedded Linux, close a loop over CAN — was therefore exercised once at minimum stakes first, on a single-motor inverted pendulum.

The environment is a MuJoCo pendulum wrapped in a Gymnasium interface. Its observation is deliberately *the hardware telemetry and nothing else*: [position (turns), velocity (turns/s), torque (N·m)], exactly as the drive reports it. The policy outputs a torque command clipped to $\pm 1 \text{ N·m}$. Success

means reaching the upright band within 5s, then holding ± 0.1 turn and ± 0.1 turn/s to the end of a 20s episode. Training is SB3 PPO with a curriculum. The export bakes the observation normalisation statistics *into* the ONNX graph, a detail that returns as a hard requirement in Section 3.8.9. The policy then runs on a Raspberry Pi under `onnxruntime`, closing the loop on a moteus drive over CAN at 50 Hz with a watchdog. The sim-to-real transfer succeeded, and the exercise bought exactly what it was designed to buy.

- The **train→ONNX→embedded-inference path** existed and was debugged a year before the biped needed it. Transport initialisation on the CAN adapter was already non-trivial and cost real iterations at this stage, where iterations were cheap.
- The discipline of **building the observation from raw hardware telemetry** rather than from simulator conveniences. That discipline reappears later as the “no privileged observations” rule of Section 3.8.12.
- Early evidence on the **compute question** of Section 3.2.4. The BeagleBone-class board originally envisaged was rejected partly on the experience gained here.

The rig also survives as a candidate student laboratory exercise. It demonstrates the entire RL-for-hardware loop with one motor and one afternoon of training.

3.8 RL policy design and validation

The control policy went through three distinct action-space designs over the project. This section documents all three, the reflex structure that ended up wrapped around the final one, and the results obtained so far. The learning algorithm was held fixed throughout: PPO from Stable-Baselines3, an actor-critic MLP, trained on CPU-parallel MuJoCo environments. The comparisons are therefore about the *action representation*, not about the optimiser.

3.8.1 Attempt 1: direct joint torque

The first framework commanded joint torque directly. A fixed torque scale on the actuators mapped the policy output to motor setpoints. The work used a simplified bipedal robot, because the robot developed in this thesis was still being designed. The formulation is the textbook one, and it is what most legged-locomotion papers use in simulation.

It was abandoned for two reasons.

It did not converge. The training history from that period is an unbroken sequence of reward-shaping attempts. The fall penalty rose because the robot learned to belly-flop. The alive bonus changed because the robot learned to dawdle. The feet-air-time thresholds were re-tuned because the robot learned to vibrate in place. After a few iterations of that whack-a-mole exploit patching, the underlying problem became clear. A raw torque action space forces the policy to synthesise two things from scratch at every control step: the gravity compensation that keeps the robot standing, *and* the periodic structure that makes it walk. The reward landscape between those two is dominated by ways of falling over that convinced the reward the robot was doing well.

More decisively, it was judged undeployable. The CubeMars drives expose a servo mode whose internal position and velocity loop runs on the drive itself. Commanding torque means bypassing that loop and streaming current setpoints over CAN, from Python, on a Raspberry Pi. The timing budget was estimated rather than measured, and the estimate was pessimistic on three counts. It assumed the achievable loop rate would be low and jittery. It assumed the CAN bus could not carry the traffic needed to go faster. And it assumed that a missed deadline in torque mode leaves the drive holding the last commanded torque blindly, which on a loaded leg

integrates into a runaway rather than a pause. Only the third assumption still holds. On top of that, the actuator’s output-side torque constant was unknown, so the commanded newton-metres would have been approximate in any case.

The two arguments pointed the same way. The policy class that most needs high control bandwidth and an accurate actuator model appeared to be exactly the class for which neither was available. Torque control was dropped, and position control was adopted in its place.

The measurement campaign partially reverses this decision. The reversal is one of the clearest results of the project, because instrumentation overturned a design decision that had already been acted on. Every quantitative premise of the bandwidth argument was measured a year later (Section 3.5.6). The first two premises were both wrong, and wrong in the same direction.

- The Raspberry Pi holds a full 200 Hz control loop — drain both CAN buses, run the policy forward, write six commands — at a jitter of 0.141 ms standard deviation and with **half its time budget spare**, while the web interface competes for CPU. A 400 Hz loop also holds. Policy inference costs 117 μ s, or 2.3 % of a 5 ms budget. Far from being marginal, 50 Hz is reproducibly *worse* than 200 Hz.
- The CAN bus sits at 26 % of one 1 Mbit line, and the drives already broadcast their state at exactly 200.0 Hz whether asked to or not.

The compute and the bus were the two things assumed to be the constraint, and both hold ample headroom. The actual bottleneck lies *inside the servo mode that was kept instead*. The drive’s own position loop rolls off at 0.8 Hz (Section 3.5.3), and that is the single largest sim-to-real gap in the project.

The current direction is therefore SET_CURRENT commands, with the position PD closed on the Pi at 200 Hz, and a torque-source actuator carrying a real torque–speed envelope in simulation. That is the Cheetah-lineage standard, the drives support it, and the timing measurement shows the Pi has the budget for it. The change is not a full return to Attempt 1. The policy would still emit position targets, and the PD loop that closes them would simply move from the drive to the Pi, where its bandwidth is known and its gains are writable. What is reversed is the premise, not the action space.

3.8.2 Attempt 2: joint position targets (PD)

The second design made the action six *position* targets, one per actuated joint, at 50 Hz. The knee and the ankle follow passively, through the parallel linkage and the ankle spring. In simulation these targets drive MuJoCo `<position>` actuators, with gains and force ranges taken from the CubeMars datasheets: **kp** 120, **kv** 4 and 72 N·m at hip roll, and **kp** 200, **kv** 5 and 170 N·m at cam and thigh. On hardware the action maps one to one onto what the drives accept.

What worked. Standing and balancing converged, after a fair amount of effort and reward shaping. The contact model was switched to a point foot, in which only a 2.5 cm sphere at each toe collides. A policy therefore cannot rest a shin on the ground and damage the real robot. After that switch the policy held full-length episodes on two toe points. The switch also surfaced a floor-penetration exploit. With only the toe sphere colliding, the non-colliding foot and shin meshes could pass *through* the floor. The first apparently perfect balancing policy was largely exploiting that, by sinking its centre of mass below the floor into a stable equilibrium. The fix terminates the episode on either of two conditions: an upper-hemisphere toe contact, meaning the sphere has sunk, or any foot or shin vertex below the floor plane.

What failed: skating. Walking did not converge. After 32 million steps the policy reached a perfectly healthy-looking reward curve while *sliding* rather than stepping. Both feet were grounded on 57 to 66 % of steps, at a duty factor of 0.74 to 0.83. The median “swing” lasted one control step, which is contact chatter at 5 to 10 touchdowns per second. In-contact toe slip of 0.11 to 0.24 m/s accounted for essentially all of the translation, and the mean forward velocity was -0.09 m/s under a $+0.5$ m/s command.

Three separate causes were found and fixed. First, nothing priced foot motion while in contact, so a slip penalty was added, with a grounded predicate that survives the micro-hop and ghost-drag dodges. Second, the air-time term could only ever punish stepping, because credit was paid at touchdown and never lifting scored zero. That term was made one-sided and capped, then paired with a per-foot stance-time cap and a swing-clearance term, to give PPO an immediate gradient toward lifting a foot. Third, the contact physics silently made skating free: `condim=3` disables torsional and rolling friction, which turns the toe spheres into ball casters.

Two reward-plumbing bugs compounded the problem. Observation normalisation was also normalising the *reward*, dividing the carefully balanced raw scale by a return standard deviation of about 110, so a 200-point fall penalty reached the optimiser as -1.8 . And the entropy coefficient on a clipped Gaussian policy amounted to free entropy, so the action standard deviation ratcheted up until training behaviour and deterministic evaluation were two different controllers.

The lasting methodological lesson is the first one. **The system will happily exploit any loophole in the reward function.**

Why the design was abandoned. The fundamental problem survived every fix. A per-step position policy must rediscover the periodicity of a gait from scratch at every control step, because nothing in the representation says that locomotion is cyclic. A plant change compounded that. Updating the six motor masses, which total 7.1 kg and more than half the robot, and re-settling the stance keyframe under gravity, moved the plant far enough that the earlier standing policy no longer transferred. The walking policy that had been improving slowly stopped improving at all. Rather than re-tune the same representation against a heavier plant, I changed the representation.

3.8.3 Attempt 3: a central pattern generator tuned by the policy

The third design puts the periodicity into the action space itself. The policy no longer emits joint targets. It emits the *coefficients of a Fourier series* that describes one gait cycle, and the environment reconstructs the joint targets from those coefficients at the current gait phase. The result is a central pattern generator whose shape is learned rather than hand-authored. The network no longer has to invent periodicity. It only has to choose which periodic signal it wants. The BioRob group’s work on the same idea was discovered after this design emerged.

Formulation. For the two sagittal joints of the left leg (cam and thigh, the pair that drives the parallel 4-bar and therefore propulsion), the deviation from the nominal stance posture at gait phase φ is

$$\Delta q_j(\varphi) = A_j \left[w_0 a_0 + \sum_{k=1}^N w_k (a_k \cos k\varphi + b_k \sin k\varphi) \right], \quad w_k \propto \frac{1}{\sqrt{k}}, \quad (3.1)$$

with $N = 3$ harmonics, which is seven coefficients per joint. The harmonic weights w_k are normalised to sum to one, which bounds the waveform. The $1/\sqrt{k}$ schedule was chosen deliberately over the more obvious $1/k$. A $1/k$ schedule biases the series so strongly toward its fundamental

that every gait it can express is near-sinusoidal, which is to say walking. The push-off of a running stride is impulsive and needs high-harmonic authority. The result is clipped to $\pm A_j$ around the nominal posture. That bound is *relative*, because the nominal posture is always validated and assemblable, and the coupled 4-bar assembles over only a diagonal band of the joint box.

The phase φ advances at a frequency that is itself part of the action, so the policy chooses its own cadence. The right leg is not emitted at all. It is constructed as the mirror of the left, half a cycle out of phase, as $\Delta q_{R,j}(\varphi) = -\Delta q_{L,j}(\varphi + \pi)$. Mirroring halves the action dimension, guarantees an alternating gait by construction, and matches the convention the hand-authored gait demo already used. It carries one consequence worth flagging, because that consequence cost time later. A mirror-symmetric gait can only travel in a straight line. Steering therefore had to be added afterwards, as two explicit asymmetry channels — differential stride amplitude and differential stance width — rather than emerging on its own.

First version: per-cycle updates, and why it was not enough. The initial implementation made one environment step equal one full gait cycle. The policy chose a gait, the environment ran that gait to completion, and the macro reward was the mean of the per-step rewards. An episode then held roughly 30 decisions instead of 1000. The design was immediately better than the per-step PD policy. A hand-crafted elliptical foot path pushed through the same reconstruction walked at 0.357 m/s open-loop, against the PD policy’s 0.01 m/s. The first trained policy then walked properly at about 0.5 m/s, with no skating signature at all. The structural prior worked exactly as intended.

The design was nevertheless slow, and it fell over as soon as the pitch degree of freedom was released. A literature review at that point returned an unambiguous verdict: **no published system achieves running with per-cycle-only parameter updates and no intra-cycle feedback.** Every successful periodic parameterisation keeps a per-step channel. PMTG modulates the trajectory generator and adds per-step residual corrections. CPG-RL re-selects the oscillator parameters at 100 Hz. The same review found a genuine semi-Markov bug in this formulation. Taking the *mean* of the per-step rewards over a macro-step, with a fixed discount per macro-step, makes the total return scale with cadence. The effective horizon then varies between 4.8 s and 28 s, depending on the gait frequency the policy happened to choose. The review flagged two missing observations as well. Base velocity was not observable at all, even though the reward maximises it. Distance-to-the-line was also missing, which left the value function partially observed and made braking unplannable.

Final version: per-step re-parameterisation plus residuals. The current stack keeps the Fourier gait but re-emits the entire specification at *every* control step, and adds a per-step correction channel on top. The 24-dimensional action is:

dims	channel	role
14	Fourier coefficients	cam + thigh gait shape, Equation (3.1); mirrored to the right leg
1	gait frequency	cadence, 0.5–4.0 Hz; the phase integrates it
3	hip-roll reflex gains	k_p , k_d , bias of the lateral balance reflex
6	per-step residuals	direct corrections on the six PD targets, ± 0.08 rad

Table 3.12: The per-step action. The first 18 dimensions are the gait *specification*; the last six are the fast-feedback channel.

Rewriting the gait specification mid-cycle is legal but priced. A penalty proportional to $\sin^2(\varphi/2)$ bills every change to the specification, so a rewrite is free exactly at the cycle boundary and most expensive mid-stride. The residuals are deliberately *never* billed by that penalty, because they exist to be per-step. A one-off stumble correction should not have to pay for rewriting the whole gait. The separation is therefore clean. The Fourier channel is the gait, and the residual channel is the correction. A separate magnitude penalty keeps the residual from quietly becoming the controller.

The observation holds 55 values stacked over 5 frames, or 275 in total. It carries motor position, velocity and torque; the IMU gravity direction and gyro rates; base velocity; $\sin \varphi$ and $\cos \varphi$; a task channel with the run/stop flag and the distance remaining; and the previous action.

Reward. The sprint objective is a single dash: stand at the line, run, stop. One insight carries the design, and it is worth stating because it is not obvious. **A per-step speed reward cannot price time.** Integrating $w v_x$ over a fixed course gives $w \cdot \text{distance}$, whatever the pace, so a fast run and a slow one score identically. What prices time is a *constant per-step clock cost*, which integrates to $-w_t T$. Correspondingly, the alive bonus must be exactly zero, because paying per second of existence rewards dawdling. The break-even pace is the ratio of the two, at 0.25 m/s.

Three groups of terms sit around that core. The anti-skate terms are inherited from the PD era. A Siekmann-style phase-gated contact schedule makes each foot pay for ground contact during its expected swing window, using the action’s own gait clock. As the stance ratio ramps from 0.65 down to 0.42, the two swing windows begin to overlap, which demands a flight phase explicitly. That is the one term which asks for running rather than for fast walking. The efficiency terms price torque, motor velocity and positive mechanical power. They ramp in over 60 million steps, so that they cannot smother gait emergence early.

Adversarial review of the reward code found four exploits, all fixed before the long runs. A stop-phase income was net-positive per step, and therefore farmable forever by twitching the hold timer. A one-sided velocity clip made shuttling back and forth in front of the finish line strictly better than crossing it. And a cold-start suicide made diving at -100 value-optimal for any policy that could not yet locomote, because the always-on penalties for a standing robot sum to about -2.2 per step. The last was fixed by flooring the per-step total, not just the individual terms.

Curriculum over base degrees of freedom. Asking a fresh policy to solve balance and locomotion simultaneously is too much, so the base is released progressively instead. The model’s free joint was replaced by six explicit scalar joints — x , y , z , roll, pitch and yaw — plus six equality constraints that can be activated individually. Any subset of the base degrees of freedom can therefore be rigidly railed at reset. Milestone **m1** frees forward translation only, with height railed at a randomised ride height drawn from a measured lookup table. Milestone **m2** adds height, **m3** adds pitch, **m4** adds lateral translation and **m5** adds roll. Milestone **m6** is the real plant. Rigid equality constraints were chosen over clamping the state after each step, precisely because they produce the correct reaction wrench *during* the step, which keeps the leg torques faithful. Critically, the observation and action dimensions do not change across milestones, so every stage warm-starts from the previous stage’s checkpoint.

3.8.4 Reflexes: what is learned and what is hard-wired

Not everything in the controller is a Fourier series, and the split is deliberate. A Fourier series over gait phase is *feedforward* by construction. It is a function of where the robot is in the cycle, not of what the robot is doing. Balance is not a periodic problem but a feedback problem, and

encoding it as a phase-indexed trajectory would make it blind. Three feedback channels therefore sit outside the CPG.

Lateral (abduction) reflex — learned gains, fixed structure. The hip-roll joints are not driven by the gait generator at all. They are driven by a linear reflex on base roll,

$$u = k_p \phi_{\text{roll}} + k_d \dot{\phi}_{\text{roll}} + b, \quad q_{\text{hip roll},L} = q_L^{\text{nom}} + u, \quad q_{\text{hip roll},R} = q_R^{\text{nom}} - u, \quad (3.2)$$

evaluated at every control step. The left and right roll axes are mirrored, so opposite signs in joint space move both feet the same physical way. That is the lateral shift of the centre of mass the reflex is for. The three gains (k_p, k_d, b) are *emitted by the policy*, so the controller is learned, but only three numbers of it are, rather than a free trajectory. That decision is what made the original per-cycle formulation tractable at all: a feedforward CPG for propulsion, plus a feedback reflex for balance, instead of a per-step policy with free hip roll. It was kept in the per-step version because it works. Nothing assumes which sign stabilises, and the policy discovers that. The reflex is necessarily inert while roll is railed, across milestones **m1** to **m3**, where the gains are unidentifiable. It becomes live only at **m5**. That is expected behaviour rather than a bug, and it is worth knowing before reading those training curves.

Sagittal (pitch) reflex — hand-tuned, not learned. A fixed PD on base pitch and pitch rate drives a *symmetric* fore/aft thigh offset,

$$u_p = -\text{clip}(k_p^\theta g_x + k_d^\theta \dot{\theta} + b^\theta, \pm u_{\text{max}}), \quad q_{\text{thigh},L} += u_p, \quad q_{\text{thigh},R} -= u_p, \quad (3.3)$$

where $g_x \approx \sin \theta$ is the fore/aft component of the measured gravity direction. The thigh axes are mirrored here too, so opposite signs move both feet the *same* physical way. When the robot pitches nose-down, both feet shift forward, under the falling centre of mass. That is capture-point control in its crudest form, and the gain scales follow that reading: $k_p \approx h_{\text{CoM}}/\ell \approx 1$ and $k_d \approx \sqrt{h/g} \approx 0.3$.

Why this reflex is *not* learned is worth spelling out, because it is the opposite choice from the hip-roll reflex. It was introduced when the pitch-free milestone was collapsing in 1.5s, and a learned gain has no useful gradient in that regime. The policy has to survive long enough to experience the correction before it can learn to tune it, and it was not surviving. A hand-tuned prior breaks that circularity. The gains were set by a grid search on the pitch-free plant. Values of $k_d \geq 0.3$ oscillate through the one-step action delay and the target smoothing filter, and an authority clip above 0.35rad starts fighting the gait itself. Those two bounds fix $k_p = 2.0$, $k_d = 0.2$ and a clip of 0.25rad. Measured on a pitch-kick arrest test, the reflex reduces the final pitch excursion by a factor of 2.6 to 6.7 in both directions. It also lifts early-retraining survival from about 18 to about 80 control steps.

That last number is the point. The reflex is a **stabilising prior, not a controller**. The plant with height free cannot stand at zero action at all, because holding a ride height requires an active gait, so it collapses in about 1.3s whatever the pitch does. The reflex does not fix that. It buys the policy enough surviving episodes to find a pitch-neutral gait itself. An early acceptance gate of “zero action stands for 20 seconds” was simply wrong for this plant, and the kick-arrest test replaced it.

Residuals — the fast channel. The six per-step residuals are the third feedback path, and the one the literature is most emphatic about. They let the controller respond to something unmodelled within a single control step, without rewriting the gait. They are kept small, at 0.08rad, raised to 0.20rad once the control rate moved to 200Hz. They are also magnitude-penalised, so that the Fourier prior stays dominant.

All three channels are implemented in pure NumPy, in the same module that reconstructs the gait, with no dependency on the simulator. That is not incidental. It is what allows the identical reflex code to run on the Raspberry Pi against a real IMU, instead of being re-implemented and re-tuned at deployment time.

A negative result: the emulated ankle reflex. One more reflex was built and rejected. The ankles are passive springs with no actuator, so a fixed PD was implemented to apply a pitch-restoring *torque* at the ankle joints, gated on that foot being grounded. The result is an emulated actuated ankle, and a shippable controller if a real ankle motor were ever added. At 40 N·m of authority it *destabilised* the running gait outright, collapsing episodes to 28 steps, because a phase-blind torque fights the gait it is trying to stabilise. At a gentle 12 N·m it neither destabilised anything nor solved anything, and gave only a small lift. The negative result was still informative. It was evidence that ankle authority was the right lever, and that is what motivated the passive-stiffness experiment described next.

3.8.5 The pitch-balance wall, and how it was broken

Everything through milestone m2 trained well, with forward translation and height free and attitude railed. Releasing pitch at m3 produced the hardest failure of the project. Episodes plateaued at roughly one second, which is the passive fall time, and stayed there. Two full sessions of work went into the failure. What was ruled out is worth recording, because the null results are the useful part.

What was tried	Outcome
Reward/curriculum easing (7 variants: no flight demand, efficiency off, low speed cap, more authority, height shaping, competence-gated hardening)	All plateaued at the same episode length. The reward was not the bottleneck.
Decaying external pitch assist (a spring-damper on the base pitch joint, faded to zero)	<i>Crutch dependence.</i> In-training episode length looked excellent; with the assist forced to zero at evaluation the policy was <i>worse</i> than the plateau, and drifting backwards.
Anti-crutch penalty on the assist torque the policy provokes	No improvement at low weight, worse at high weight.
Pitch “slow motion” via extra armature on the base pitch DOF, faded out	Marginal gains that did not survive the fade.
Raising the control rate 50–200 Hz and increasing residual authority (0.2–0.4) and filter bandwidth	All plateaued at ≈ 1 s. Since the learned residual gives <i>more</i> foot-placement authority than any fixed heuristic could, this implicated the plant rather than the control law.
Emulated actuated ankle (fixed torque reflex)	Strong version destabilised; gentle version marginal.
Lowering the centre of mass by 6–12 cm	Slightly <i>worse</i> .

Table 3.13: The pitch-balance failure: everything that did not work.

Two methodological lessons came out of that table. First, **any training aid that holds the robot up will be leaned on.** The in-training curve then looks wonderful while the unaided policy gets worse, and only an evaluation with the aid forced to zero tells the truth. That deception caught me three times before assist-off evaluation became protocol. Second, when every software lever saturates at the same value, the constraint is not in the software.

The diagnostic that cracked it. Rather than try a ninth lever, I instrumented the failure. A tool rolls out a policy and plots, per foot, the fore/aft offset between the toe at touchdown and the whole-robot centre of mass. The result was unambiguous. The base pitch ramps monotonically forward into a pure topple. Meanwhile the feet land 8 to 9 cm *behind* the centre of mass. Only 11 % of touchdowns landed ahead of it, and they fell progressively further behind as the body ran away forward. The policy had a perfectly serviceable forward-running gait and had never learned the *capture step*, which plants the foot ahead of the centre of mass to catch a forward fall. The robot was, quite literally, running out from under itself.

The fix was a plant change. The diagnostic pointed at the passive ankle. Stiffening the ankle spring by roughly a factor of twelve, from 28.65 to 350 N·m/rad, gives the passive foot the stance pitch-restoring authority the plant lacked. The threshold is clean. A stiffness $k \leq 200$ barely helps, while $k \geq 350$ breaks out of the plateau and climbs steeply. After the fix, the same diagnostic reports 67 % of touchdowns landing ahead of the centre of mass, with the base pitch pinned near zero.

One implementation detail is essential, and getting it wrong cost a wasted run. The standing ankle sits at about ± 0.25 rad while the spring reference sits at ± 0.7 rad, so the spring carries about 12.8 N·m of *preload* at stance. Raising the stiffness naively balloons that preload. The foot then slams to its spring reference and the robot flips over. The reference has to be shifted at the same time, so that $k(q_{\text{stand}} - q_{\text{ref}})$ is preserved. Only the restoring *gain* then rises, and the posture is unchanged.

The result also *overturned* an earlier recommendation of my own. After the emulated-ankle experiments I had concluded that the robot needed a policy-controlled actuated ankle: a motor, two more action dimensions, and a full retrain of the milestone ladder. The stiffness experiment shows that a passive spring suffices, which is dramatically cheaper. As Section 3.6 notes, the static load analysis independently arrives at the same 350 N·m/rad threshold.

That left one question open: whether the real ankle could be built that stiff. The answer, when it was measured, invalidated the result.

The measured spring, and what it does to this conclusion. The fabricated ankle measures $k = 41.4$ N·m/rad with **no preload**. Neither number matches the model. A spring at $k = 350$ will not be built, and the 12.8 N·m of stance preload the simulation carried does not exist on the robot.

The second discrepancy matters more than the first, and that is the genuinely useful finding here. The preload, not the stiffness, was doing almost all of the work. At the same k , the modelled spring sags 19 mm at stance with its preload, and **145 mm without it**. That reframes the whole ankle question. The parameter the study spent its compute sweeping was not the parameter that mattered.

Statics disposes of the real spring before any learning is involved. The easiest possible load case is 0.5 body weights per foot, in double support, against a 3.5 BW running requirement. Even there, the no-preload spring deflects 26.6° , drops the robot 145 mm, and puts the heel 11.9 mm into the floor at the reset pose. Contact forces stay well behaved throughout, so the collapse is

genuine loss of load capacity rather than a solver artefact.

Everything in Table 3.14 was therefore obtained on an ankle that does not exist. The balance result of this section is real as a statement about what the *plant* needed. It is not a statement about what this robot does.

The live candidate is now neither the fabricated spring nor a stiffer one. It is a tension-only strut, taut at the flat-foot angle and buckling in compression. It stands like a welded ankle, at 0.08° of deflection and 0.8 mm of sag, and deleting the spring assembly removes 249 g from each shin. Table 3.11 independently favours that change. At -0.93 (m/s)/kg on distal mass, 249 g per leg is worth roughly $+0.5 \text{ m/s}$ of top speed, which is the largest single improvement available anywhere on the robot. A design decision reached for one reason turns out to be supported by an unrelated measurement, which is the closest this project comes to a free result.

3.8.6 The cadence problem and a cautionary bug

With balance solved, the remaining pathology was cadence. The policies stepped far too fast, on the order of 11 Hz, pattering rather than striding. Several external limits were tried: hard velocity caps, acceleration caps and a cadence penalty. All of them collapsed the warm-started policy instead of slowing it.

Instrumenting the trained policy’s own action channels found the cause. When the control rate was raised to 200 Hz, the learnable gait-frequency range had been widened to 0.5–50 Hz, on the reasoning that so wide a range would be “non-binding”. The map from raw action to frequency is linear over that range, so a *neutral* network output of about zero means 25 Hz. The policy simply left that output near zero and inherited a 24 Hz gait clock. The commanded thigh trajectory was reversing sign 35 times a second at $\pm 0.45 \text{ rad}$, a signal that would destroy the real actuators. Every external limit had been fighting a broken internal default. Remapping the range to 0.5–4.0 Hz, so that a neutral action means about 2 Hz, produced the best controller of the entire project (Table 3.14).

The general lesson is worth carrying. **A parameter range widened to be “non-binding” can silently become the operating point.** A network’s default output is not a neutral gait. It is whatever the middle of the range happens to decode to.

One pathology survives the fix. After the remap the gait clock slowed to about 1 Hz, but the actual *footfall* rate stayed near 6 Hz. The policy steps through the fast residual channel rather than through the clock. A ring-down test placed the passive ankle mode at 1.0 Hz, not 6.3 Hz, so the chatter is a forced oscillation rather than a resonance: the policy pumps its legs at 200 Hz to balance a top-heavy robot. Five levers failed to remove it — a residual-rate penalty, a torque-budget curriculum, a reduced pitch-reflex derivative gain, low-pass filtering of the reflex input, and heavy ankle damping — and every warm-started run stays trapped in the fast-stepping basin. The honest conclusion is that removing it needs a from-scratch retrain with cadence pressure present from step zero, or a plant change such as a lower centre of mass. On hardware, the ankle’s real mechanical damping would absorb some of it.

3.8.7 Results

Milestone	Free base DOF	Steps	Survival	Outcome
m2	x, z	180 M	53 s	runs; the reliable working regime
m3	+ pitch	600 M	39 s	completes the dash and stops, 2.6 m/s
m4	+ y	600 M	38 s	completes the dash and stops
m5	+ roll	600 M	26 s	high variance — the weak link
m6	all six	600 M	60 s	never falls; runs through the line
m7	+ pitch (cadence fix)	163 M	37 s	best controller: 2.71 m/s, very low variance

Table 3.14: Deterministic (greedy) evaluation of the milestone ladder, each stage warm-started from the previous. m7 is not a new degree of freedom but the m3 plant retrained after the gait-frequency fix of Section 3.8.6. Survival is capped at 60 s by the episode limit, so m6 is at the ceiling. **Plant epoch: CAD masses (12.83 kg), $k = 350 \text{ N}\cdot\text{m}/\text{rad}$ ankle with preload, datasheet peak torque, an idealised $\approx 35 \text{ Hz}$ drive.** None of those four match the measured robot; see Section 3.8.8.

The headline result is that **the fully free 6-DOF plant runs and does not fall**. That is 60 seconds of simulated running, at the episode limit, with zero variance across evaluation episodes, at 2.6 m/s. Two caveats belong next to it. Milestone m6 balances but runs *through* the finish line without executing the stop, which is a task-completion defect rather than a balance one. Milestone m5 still topples on a fraction of episodes and needs more training. Against the capability bound of Section 3.6, 2.6 m/s is about 70 % of the credible-cadence ceiling, and already above the thermally sustainable figure. On the corrected plant of Section 3.6.6 it sits at the burst ceiling, and at twice the sustained one.

An A/B on the generator itself. The Fourier series is one of several ways to parameterise a periodic gait, so it was compared directly against a phase-amplitude coupled oscillator. That oscillator is the kind used in the biologically inspired locomotion literature, and its foot trajectory maps through a measured inverse-kinematics table. Sixteen runs shared one plant, one reward and one schedule, and only the generator was swapped. The verdict is that **there is no clear winner**. The oscillator is more sample-efficient from a cold start on the hardest milestone. The Fourier generator is more reliable across random seeds. The single best controller measured in the whole experiment was an oscillator run: the only arm of the sixteen that ever executed the stop phase, at 2.9 m/s with a left/right asymmetry of 0.08.

One ablation in that experiment is worth reporting, because it refutes a hypothesis stated earlier in this chapter. Removing the per-step residual channel, which leaves a pure oscillator, produced *stability without locomotion*. The robot survived most of the episode but travelled 15 m of 72, at 0.25 m/s, with the worst cadence in the entire set. The residual channel was the suspected fast open path behind the 6 Hz footfall of Section 3.8.6. Removing it made the chatter worse rather than better, so that explanation is wrong.

Cost. Training ran on the EPFL Izar cluster. Throughput is about 2900 environment-steps per second on a V100 node, which is about $1.9\times$ the CPU-only rate on the same node. The gain is not down to network size, and that is worth recording. Physics always runs on the CPU, because this stack has no GPU physics backend. Putting the tiny policy network on the GPU simply stops PyTorch from stealing cores from the parallel MuJoCo workers. At that rate a 600 M-step milestone is a multi-day job, which is why the warm-start chain across milestones

matters as much as it does.

3.8.8 Which of these results survive the plant correction

Section 3.5 invalidates the plant that every number in Table 3.14 was measured on. Deleting the table would be less useful than stating precisely which conclusions depend on the plant and which do not, because that distinction *is* the sim-to-real argument.

Survives a plant change	Does not
The action-space comparison: torque, per-step position, and periodic generator with residuals, in that order of tractability.	Every absolute speed and survival time in Table 3.14.
The reward-exploit register (Section 3.8.2) and the general lesson that reward curves cannot certify a gait.	The claim that the fully-free plant “runs and does not fall”. It ran, on a plant with an ankle that will not be built.
The base-DOF curriculum, and warm-starting across stages with fixed observation and action widths.	The ankle-stiffness recommendation, which Section 3.8.5 replaces.
The gait-frequency bug and its general lesson about non-binding parameter ranges.	The 2.6 m/s figure as evidence of hardware capability.
The capture-step diagnostic and the finding that foot placement lagged the centre of mass.	Any claim of readiness for deployment.

Table 3.15: What the plant correction of Section 3.5 does and does not invalidate.

The pattern is that the *methodological* results transfer and the *performance* results do not. That conclusion is unsurprising in hindsight and expensive to learn. Roughly 3 billion environment-steps of training were spent on a robot 18 % too light and 18 % too strong, with an ankle spring an order of magnitude too stiff and an actuator an order of magnitude too fast.

3.8.9 Validation status and known gaps

Stating the validation status plainly matters more than the results above, because the gap between the two is the work that remains.

Sim-to-sim is exercised continuously by the milestone ladder. Each stage is a different plant, with different constrained dynamics, and every stage loads the previous stage’s weights. A policy that only worked by exploiting a particular rail therefore fails visibly at the next rung. The ankle-stiffness and centre-of-mass experiments use the same mechanism deliberately, as a plant-sensitivity probe. The drive-bandwidth correction of Section 3.5.3 is its most severe instance: a converged walker collapsed from an episode length of 1448 to 251, with only the ankle and the drive changed.

No trained policy has run on the robot. That is the precise statement, and it is narrower than “nothing has run on hardware”. A great deal has. The hand-authored fixed gait plays on the real motors. The web interface drives all six joints under safety limits. The workspace calibration and the system-identification excitation runs are recordings taken from the robot, and every measurement in Section 3.5 came off it. What has never happened is a learned controller closing a loop on the physical robot.

The sprint policies in Table 3.14 are not deployable as they stand, independently of the plant correction. They observe two privileged quantities that do not exist on the robot. One is base linear velocity, which would require a state estimator that has not been built. The other is the distance remaining to the finish line. A separate policy lineage was therefore started, for a drivable demonstrator. That lineage removes both privileged quantities, so velocity must instead be inferred from a 200 ms strided observation history. It adds per-episode plant randomisation over mass, inertia, friction, servo gains, joint damping, ankle spring stiffness, gravity tilt and actuation delay. It adds a sensor-noise model, including the accelerometer leakage that makes a real IMU’s “down” swing by several degrees during push-off. And it adds the steering channels the mirror-symmetric gait lacks. The lineage cannot warm-start from the sprint milestones, because both the observation and the action dimensions changed, so it carries a separate training cost. Section 3.8.12 reports its results and the diagnoses that produced them.

Domain randomisation ranges are now set from measurement rather than from taste, wherever a measurement exists. Ground friction is sampled over 0.35 to 0.60 against a measured $\mu = 0.84$, from a 40° slip angle. That range sits deliberately well below the real value, because the failure mode being guarded against is planting a foot that then slides. Joint homing error is sampled at 5° against a calibration accurate to 1 or 2°, as a safety envelope. Where no measurement exists, the range stays wide and is flagged as such. Link inertia tensors are randomised at $\pm 25\%$ precisely because Section 3.5.5 could not resolve the mass distribution.

Three known gaps remain, recorded deliberately rather than discovered later. There is no policy export path and no inference loop on the Pi, which is the critical path for everything else. The actuator model still gives each motor an independent torque limit and speed limit, rather than a coupled torque–speed envelope, which makes the simulation optimistic exactly in the high-speed regime the project cares about. And first hardware bring-up remains planned with the base fixed and the legs in the air. That configuration validates wiring, direction conventions, joint limits and gait shape, but with no ground contact it says nothing about balance or transfer.

The five subsections that follow report what has happened since this status was written: a generator A/B (Section 3.8.10), the ankle resolved as a build decision (Section 3.8.11), the drivable lineage’s descent onto the measured plant (Section 3.8.12), a classical baseline built to test whether learning was ever the bottleneck (Section 3.8.13), and the rebuilt robot (Section 3.8.14).

3.8.10 A second generator: explicit-oscillator CPG versus the Fourier series

The Fourier parameterisation of Section 3.8.3 is one member of a family. The other canonical member is the explicit limit-cycle oscillator of the Ijspeert school, as used by CPG-RL. The literature gave no basis to prefer one over the other, so the question was measured instead: sixteen cluster jobs, one plant, one reward, one schedule, and *only the generator swapped*. Building the oscillator arm forced a foot-space formulation with a measured inverse-kinematics lookup table. Constructing that table surfaced four plant facts worth recording. From the stance pose, *both* cam and thigh move the toe mostly fore–aft, and either sign of cam *shortens* the near-straight leg. The intuitive “thigh swings, cam lifts” mapping is therefore simply wrong, and it produces no step. The 4-bar has a folded solution branch that must be filtered out. The inverse-kinematics inversion must propagate continuity between cells, or it flips branches mid-swing. And reachable foot lift at mid-stride is only about 8 cm, because the leg is near-straight there.

Condition	Oscillator (CPG)	Fourier
cold m3 (from scratch, pitch free), 133 M m2 (pitch railed)	ep_len 408, return +4 best single controller of the experiment: 73.5 m dash, 2.9 m/s, <i>the only arm that executes the stop phase</i> , L/R asymmetry 0.08	ep_len 237, return -89 fastest arm of the experiment (3.30 m/s, 5/5 episodes) but never executes the stop
m2→m3 warm start	3/5 episodes cross at 2.40 m/s	5/5 cross at 2.45 m/s

Table 3.16: Generator A/B at matched budget. No clear winner: the oscillator is more sample-efficient cold, the Fourier series more reliable across seeds; both show the same bimodality on the warm-started pitch-free plant.

Two results stand out beyond the tie.

First, the **pure oscillator with the residual channel removed produces stability without locomotion**. It survives 9700 of 12 000 steps, but travels 15 m of 72 at 0.25 m/s, with the worst foot-scuffing cadence of all sixteen arms. That result *refutes* the standing hypothesis that the per-step residual channel was the open fast path causing the 6 Hz footfall chatter of Section 3.8.6, because removing the channel made the chatter worse rather than better. It is also the empirical form of the literature’s verdict that a per-step channel is mandatory.

Second, the full base-DOF ladder was subsequently run on the oscillator generator. It **cleared m1 to m4, with a 100 m dash at m2 and 2.95 m/s at m3, and walled hard at m5, which frees roll**. Episode length there was 337 against the Fourier ladder’s 3213 at matched budget, and the plateau was confirmed at $2.5\times$ budget. On the roll axis the Fourier generator succeeds where the oscillator does not. Both share the identical 3-parameter roll reflex, so the difference lies in the generator itself, and it points future investigation at lateral authority. The pragmatic conclusion adopted was to keep the Fourier generator for the deployment lineage, and to use the oscillator result as tiebreak evidence that the action-space *family* is right.

Getting this A/B to run also produced a curriculum lesson expensive enough to state as a rule. One milestone in the ladder took six restarts, five of them self-inflicted. The terminal bug was a competence gate that held *both* the difficulty ramp and the efficiency-penalty ramp. For a policy that never reached the gate, the efficiency terms stayed pinned at exactly zero for the entire run. A run that looked like “the robot cannot learn this” was actually “the reward is not the one we designed”. What settled it was not another hypothesis but a field-by-field configuration diff against the last run that worked. **Never gate the ramp that is supposed to rescue competence on competence itself, and diff configurations before theorising.**

3.8.11 The ankle, revisited as a build decision

The 350 N·m/rad ankle-stiffness result of Section 3.8.5 answered a simulation question. The hardware then asked a different one. The spring actually fabricated measures $k = 41.4$ N·m/rad with *no preload* (Section 3.5.4), and a spring wound to 350 will not be manufactured. The question became what should be *built*: the real soft spring, or no spring at all. No spring means a rigid strut, which would also delete the 249 g spring assembly from each shin, on the axis the sensitivity analysis prices at -0.93 (m/s)/kg.

The soft no-preload spring is eliminated by statics alone, before any learning. The easiest load case there is holds the base and applies 0.5 body weights per foot in double support, against a 3.5 BW running requirement. Even there the as-built spring deflects the ankle 26.6° past flat and drops the robot 145 mm. It also rotates the foot so far that the heel carries 208 N

while the toe is driven 11.9 mm *into* the floor at the reset pose. No controller fixes that. A policy chooses where to put a foot. It cannot make a spring hold seven times its load. The instructive split is preload against stiffness. The same $k = 41.4$ sags 19 mm *with* the original preload and 145 mm without it, so **the preload, not the stiffness, was doing $7.6\times$ of the work** in the shipped ankle.

The strut wins the learning screen. The RL comparison ran cold m3 on the corrected plant. The compression-capable strut arm reaches episode length 2307 in *one third* the steps the fully welded ankle needs, at 29 M against 73 M, and walks $3\times$ more symmetrically. Given time, the welded arm completes the 57 m dash at 2.55 m/s in 5 of 6 episodes. A tension-only strut, with no compression capacity, manages 0.6 m and stands on one leg. The compression capacity is the entire story, because the arms are statically identical otherwise.

And the as-built part fails by arithmetic. The fabricated strut is a solid PETG rod, 500 mm long and 3 mm across. Its slenderness is $L/r = 667$ and its Euler buckling load is **0.24 to 0.33 N**. That is thirty-five times below the 10 N the simulation arm assumed. It is also ten times less moment at the ankle than the foot's own gravity torque, so the rod cannot even hold the foot flat in swing. Its tension capacity of 283 N also falls below the 360 N walking peak. Two parts reproduce the simulated one: a 3 mm pultruded carbon rod, at $P_{cr} = 14.1$ N, 4241 N in tension and 5.7 g, or a 6/4 mm carbon tube at 181 N. **The ankle question closed as a build instruction: make it a stiff link, in carbon, and delete the spring mass.**

The instruction was executed (Section 3.8.14). The ankle is now a rigid carbon tube weighing 40 g. It removes a net 209 g per shin, taking the shin from 573 to 364 g and the robot to 14.72 kg, on the axis worth -0.93 (m/s)/kg. Taking 209 g off each shin buys more predicted speed than taking 1.8 kg off the base would.

3.8.12 The drivable-demonstrator lineage: closing on the measured plant

The deployable lineage announced in Section 3.8.9 has since accumulated results. They are the closest thing this project has to a sim-to-real verdict, short of untethered hardware. The lineage removes both privileged observations, so velocity must be inferred from a 200 ms strided history. It adds steering, and it layers on the full sensor and plant randomisation package. Its history is a sequence of measured diagnoses.

Steering was broken twice over, and both causes were in the plant model. The first policy could not turn right at all. An open-loop authority test isolated plant from policy, using a hand-built gait with roll and pitch railed, and it found two faults. First, the *model* is not left/right symmetric: two linkage anchor coordinates differ by about 1 mm in the wrong axes, a CAD-export artefact worth $+0.42$ rad/s of intrinsic yaw. Second, the two steering channels were scaled inversely to their measured authority. Differential stance width is about $6.6\times$ stronger per unit than differential stride, and the weak channel's entire authority, at 0.26 rad/s, was smaller than the intrinsic bias. The fix randomises the anchor asymmetry by ± 1.5 mm rather than symmetrising the file. The real robot will be asymmetric too, and on a perfectly symmetric plant the policy has no reason to learn the correction that transfers. After the fix, the yaw-response slope went from 0.19 to 1.22, with both turn directions reachable.

Domain randomisation was the thing breaking the policy, and that was measured rather than assumed. A fixed evaluation over 20 seeded episodes removed one adversity at a time from a trained policy. Pushes changed nothing, at $1.0\times$ the mean time between falls. Sensor noise gave $1.5\times$ and trips gave $1.8\times$. **Plant randomisation gave $19.8\times$: 95.4 s against**

a 4.8 s mean time between falls, where a clean plant gives 196 s and zero falls. The policy walked fine. Full-width plant randomisation from step zero was what kept it down, while every other axis already had a curriculum. Randomisation width now rides a gated, retreating ramp like everything else. Three evaluation-hygiene rules were paid for here. Training reward across runs with different curricula is not comparable, so always evaluate on a fixed task. Six evaluation episodes gave a $5\times$ spread on fall rate for the *same* policy, so use at least 20 seeded episodes. And a curriculum that can only advance will ride a policy down, so competence must gate the *advance*, not just the opening. One run peaked at 113 M steps and then declined monotonically for 215 M more, while the clock-driven ramps kept tightening.

The measured plant kills a working policy when applied all at once, and does not when applied as a curriculum. Every measured correction was activated together: weighed masses, 144.5 N·m torque, the strut ankle, the 0.8 Hz drive with 25 ms delay, and friction deliberately below the measured $\mu = 0.84$. The previous best walker collapsed, with nothing else changed, from episode length 1448 and reward +511 to 251 and -78. The drive bandwidth was then ramped logarithmically in hertz over 120 M steps, because the filter coefficient is violently nonlinear in bandwidth. The same policy completed the descent to the real 0.8 Hz drive and stabilised at **episode length 4938, peaking at 6833, and reward +3271**. That is three times its parent’s score, *on the real drive*, and that checkpoint is the warm-start parent for everything downstream. The residual gap to a deployable walker is honest but bounded. On the hardest configuration the current generation survives a median 9.4 s, with 3 of 12 seeded episodes reaching the 60 s cap, walking at about 0.14 m/s with a limp. That result motivated a command-range fix, now in the currently training generation: the command box asked for 0.6 m/s from a plant that tops out at 0.14 m/s, forward commands above 0.3 m/s are fatal, and *backward* is the safest direction.

The lineage also re-proved the ladder. It is the one lineage in the project that trains with all six base degrees of freedom free from step zero. It is also the one that does not converge, showing three plateaus across roughly 1.4 G steps. Meanwhile the only controller to reach 2.55 m/s on the *measured* plant did so at milestone m3, with roll and yaw railed, in 73 M steps. Everything that has ever worked on this robot used the base-DOF ladder, and an audit of the archive, covering 97 runs at m3, makes that an empirical claim rather than a preference. Two architectural responses are now in training. The first restarts the lineage *through* the ladder on the rebuilt plant. The second is an asymmetric actor–critic. The critic keeps the privileged observation, and a separately supervised velocity-estimator head replaces the privileged velocity for the actor. That head is trained on labels and detached from the policy gradient, so that PPO cannot bend it toward whatever the actor wishes were true. Its motivation is the measurement that only $R^2 \approx 0.81$ of base velocity is recoverable from the observation history.

Two bugs from this lineage belong in every RL practitioner’s checklist. A warm-started run inherited the *source checkpoint’s* RNG seed. Every warm-started “seed replicate” in the project was therefore a bit-identical duplicate, verified across two seeds and 100 rollouts with identical curves, until the seed was re-asserted after loading. The second bug was in the evaluator, which rebuilt the environment from the config’s *starting* values rather than the curriculum’s final ones. It read a 340 M-step policy at $32\times$ below its true performance. That was the fourth evaluation-restore bug of the project, and it is the reason the rule “`curriculum.json` is ground truth for anything a curriculum moved” now exists. One related point matters for reporting. Outcomes on the hard plant are *bimodal*: a policy either falls at about 2 s or survives 60 s. Unpaired small-sample comparisons are therefore noise, and one early comparison “found” a $27\times$ sensor-noise effect that does not exist. Common random numbers and survivor counts, over at

least twelve episodes, are the reporting standard for everything above.

3.8.13 A scripted baseline: how far does no learning get?

Late in the project a classical controller was written for the same plant. It combines a clock-driven foot trajectory through the measured foot inverse kinematics, once-per-swing Raibert and capture-point foot placement, low-gain pitch and roll trims, and feedforward inversion of the 0.8 Hz drive. It uses 21 hand-set gains and no training. It is injected through the environment’s residual channel, so that every plant feature — drive model, delay, randomisation, contact and termination — is byte-identical to the RL runs. Its purpose was to answer a question the RL curves cannot. When training stalls, is the *learning* the bottleneck, or the robot?

The controller’s structure is itself an argument about the plant. The foot-placement target is latched *once per swing* rather than tracked continuously. The reason is that the drive is a 0.8 Hz pole, with $\tau = 199$ ms, against an inverted-pendulum divergence constant of $\sqrt{h/g} = 311$ ms. A continuous pitch loop would need crossover around 1 Hz *through* a 0.8 Hz actuator pole, and would spend about 60° of phase before acting. Latching at swing entry instead gives the drive a whole swing, about 1.8τ , which delivers about 83 % of the commanded placement by touchdown. The same arithmetic sets the tuned cadence near 1 Hz, because at 2 Hz the drive returns 0.38 of the commanded amplitude. It also motivates the one element RL structurally lacks: explicit feedforward inversion of the drive pole, as $q_{\text{cmd}} = q + \tau \dot{q}$.

The answer is sharp in both directions. On the height-free, attitude-railed plant at m2, the scripted controller runs **60 s on all 5 seeds**, where the RL reference achieves 20.3 ± 21.6 s. It wins less on the mean than on the variance, because the RL outcome is bimodal. On the pitch-free plant at m3 it survives **1.0 s, on 0 of 5 seeds**, against the RL reference’s 1.15 s. That is *the same wall, to the decimal*.

The wall is free fall, proved by a formula with no controller in it. Modelling the robot as a rigid inverted pendulum about the toe, the passive topple time from initial lean θ_0 is $t = \sqrt{h/g} \cosh^{-1}(60^\circ/\theta_0)$. Sweeping gravity in simulation:

g (m/s ²)	9.81	6.00	3.50	2.00
predicted topple time	0.93 s	1.19 s	1.56 s	2.06 s
measured survival	1.00 s	1.21 s	1.59 s	2.25 s

Table 3.17: m3 survival time equals the analytic free-fall time across a $5\times$ change in the plant’s time constant, to 3–9 %, from a formula containing no controller term. Nothing else moves the number: drive bandwidth 0.8→35 Hz, cadence, oracle velocity, capture gain (more hurts), rear-foot support, centre-of-mass lowering, and all 21 gains were swept.

Three mechanisms compose the wall. First, the stance keyframe starts every episode with the toes 9.1 cm ahead of the centre of mass, which is a 5.67° backward lean at $t = 0$. The model’s own comment claims “feet under the CoM”, and measurement says otherwise. Correcting the keyframe buys exactly the free-fall time of the residual lean, and no more. Second, the single toe-sphere contact gives *zero* centre-of-pressure authority at any ankle stiffness. The ankle at m3 is not weak, it is disconnected. Third, foot placement is geometrically outrun. The toe hangs about 0.92 m below the centre of mass, so a pitch rate $\dot{\theta}$ sweeps the contact point forward at $L\dot{\theta}$. That sweep was measured to beat the controller’s corrective foot motion 2:1, until the ± 0.30 m fore-aft workspace saturates in under a second. Escaping m3 therefore needs one of two things. Either a corrected keyframe *plus* a foot with real fore-aft extent and an ankle strong enough

to use it, or step-on-demand triggering in place of a free-running gait clock. That is a plant conclusion rather than a controller one, which is exactly what the identical RL and scripted failures imply.

Two further findings transfer beyond the baseline. The stand keyframe’s control vector is not a static equilibrium: the position servo needs about 6 cm more commanded foot extension to hold 15 kg. RL policies discover that silently, and hand-built controllers must be told. And *feedforward beats feedback through this drive*. A constant extension offset stands for 20 s, where adding ride-height feedback drops it to 7.8 s, because a continuous loop closed through a 0.8 Hz pole against a plant diverging in 0.31 s destabilises. A methodological rhyme closes the section. The baseline’s automatic gain tuner was given the objective $t_{\text{alive}} + 3 \text{ dist}$, and it immediately exploited that objective exactly as a policy would. It drove at 1.6 m/s against a 0.3 m/s command, to farm the distance term on the pitch-railed plant. **Reward exploitation is a property of optimisation against an objective, not of neural networks.**

Why the learned controller did not win, and what that says about stability margin.

Section 2.5.2 states the theoretical expectation. The scripted controller lies inside the policy class, so with an exact plant and unbounded training the learned optimum can only match or exceed it. The measurement disagrees at both milestones, and it disagrees *for different reasons at each*. That difference is the part worth reporting. Collapsing the two milestones into a single “RL underperformed” loses the whole finding.

At m2, a margin existed and the learning did not find it. The scripted controller survives 60 s on 5 of 5 seeds, against the learned reference’s $20.3 \pm 21.6 \text{ s}$. That is not a plant limit. The plant is stabilisable, and one of the two controllers stabilises it. Three mechanisms account for the gap, and each is a statement about the optimisation rather than about the policy class.

- **The feedforward inverse is representable but not discoverable.** The scripted controller inverts the drive pole explicitly, as $q_{\text{cmd}} = q + \tau \dot{q}$. Nothing prevents a network from expressing that map, because it is linear in quantities the observation history contains. Expressing it and *finding* it are different problems. The gradient signal for phase lead is indirect. A policy is rewarded for surviving, not for leading the plant by 199 ms, and the credit for a correctly anticipated command arrives several control steps after the action that earned it. A universal policy class does not confer the credit assignment needed to exploit it.
- **Margin is not in the objective, and the variance is the evidence.** The return prices survival time and distance. It prices the distance from the state to the failure boundary nowhere. An optimiser given that objective has no reason not to park close to the boundary and collect the reward. The signature of a policy that has done so is bimodality rather than degradation: it works, or it falls. That is exactly the measured shape. A figure of $20.3 \pm 21.6 \text{ s}$ carries a standard deviation larger than its own mean, against 5 of 5 at the episode cap. **The seed variance is the margin measurement.** A controller with margin does not produce a bimodal seed distribution, and a learning curve reporting the mean will not show it.
- **“Unbounded training” is doing real work in the theoretical claim.** A milestone here is 600 M environment-steps and a multi-day cluster job (Section 3.8.7), and the project as a whole spent roughly 3 billion. That is a large budget, and it is not the asymptotic one the argument assumes.

At m3, no margin existed for either controller. Survival equals the analytic free-fall time of an *uncontrolled* inverted pendulum, to within 3 to 9 %, across a $5\times$ change in the plant’s time constant (Table 3.17). The formula that predicts it contains no controller term. The mechanisms above are the reason. With zero centre-of-pressure authority and foot placement outrun 2:1, the set of stabilising action sequences from the initial condition is essentially empty, and every

controller drawn from an empty feasible set performs identically. Reinforcement learning optimises over the space of *controllers*. It has no purchase on the space of *physics*. The theoretical margin advantage is not wrong at m3. It is vacuous, because the quantity it promises to enlarge is already zero.

Two further observations tie this to the rest of the chapter.

First, consider the mechanism one would normally use to convert the theoretical advantage into a real one. That mechanism randomises what is not known, so that the policy is optimal over a set of plants rather than over one. On this robot it was measured to *cost* more than it bought. Domain randomisation was the thing breaking the drivable policy (Section 3.8.12), and the measured plant applied all at once killed a working policy that survived the same correction applied as a curriculum. The robustness randomisation buys is paid for out of performance margin, and a plant whose margin is already near zero has nothing to pay with. On a robot with margin to spare the trade is invisible. Here it is the dominant term.

Second, margin is a frequency-domain quantity, and neither method can synthesise phase the actuator does not have. A 199 ms drive pole against a 311 ms divergence constant bounds what *any* controller can achieve, learned or hand-built. That is why the drive loop appears in Section 3.6.6 and Section 6.3 as a mechanical priority rather than as a tuning one.

The claim all of this supports is narrower and more useful than either “RL beats classical” or the reverse. **The stability-margin advantage of learning scales with how much margin the plant has to give.** On a plant with margin, learning must still be paid before it collects, in reward design that prices the boundary and in the training budget to reach the asymptote. On a plant without margin, the comparison measures the robot, and the correct response is to change the robot.

3.8.14 Postscript: the rebuilt robot, and a cap that was never there

On 2026-08-10 the hardware caught up with the analysis, and the training plant was re-issued to match. Four things changed. Three of them are the physical robot converging on the model’s recommendations. One is a modelling gap that had been open the entire project.

1. **The actuators’ speed was never capped in simulation.** Peak torque had always been limited, at the measured 61.2 and 144.5 N·m, but joint *velocity* was unconstrained. A position servo could therefore crank a joint arbitrarily fast for free. The record shows what that bought. The celebrated k350 pitch-balance winner (Section 3.8.5) “balanced” partly by pattering one foot at 11 Hz, with a peak thigh speed of 23 rad/s, *past the motor’s 22 rad/s no-load ceiling*. The cap is now set per joint, at the measured no-load output speeds: 10.30 rad/s at hip roll and 22.01 rad/s at cam and thigh. It is per joint because the two actuator families differ by $2.1\times$, so any scalar cap is wrong for one of them. The episode is one more entry in the pattern of Section 3.4.3: an unconstrained axis is an exploit waiting to be found.
2. **The ankle is now the rigid carbon tube** of Section 3.8.11. It is stiff, symmetric in push and pull, and has no buckling limit. It replaces both the soft spring and the PETG rod that modelled as tension-only.
3. **The mass moved the right way:** a net -209 g per shin, for a total of 14.72 kg.
4. **The drive was retuned from 0.8 to about 3 Hz.** The retune was measured by repeating the swept-sine of Section 3.5.3, after adjusting the boards’ position-loop gains with the vendor tool. The left thigh moved from 0.86 to 3.35 Hz and the cam from 0.85 to 2.88 Hz. The hip-roll axis was left untouched deliberately, as the in-run control, and moved only from 0.74 to 0.76 Hz. The uniform 3 Hz assumption is flagged, because hip roll uses a different actuator

and has not been re-measured, so the roll milestones may need re-baselining. The retune softens the current-mode conclusion of Section 3.5.3 without retiring it: 3 Hz is still below the 4 Hz top of the gait-frequency range.

The bandwidth bracket that justified the retune. A three-arm experiment settled whether drive bandwidth was really what stood between the robot and walking, and it ran before the ladder was relaunched. Three identical cold m2 runs used drive bandwidths of 0.8, 3 and 12 Hz, with everything else fixed, and with the measured 25 ms delay and the workspace kill retained. At 0.8 Hz, which is the robot as it was, the policy *shuffles and never walks*: duty factor 1.00, then a fall. At 3 Hz it **walks**, covering 20.75 m at 0.69 m/s with genuine per-foot flight. At 12 Hz, which is effectively the unlimited drive the whole 2.55 m/s sprint record was trained in, it is *worse* than at 3 Hz, and folds one leg into the workspace kill. Three conclusions follow. The measured retune sits, fortuitously, near the sweet spot. The old unlimited-bandwidth plant was not merely optimistic but qualitatively misleading. And the check cannot be done by replaying a policy trained at 0.8 Hz at a higher bandwidth, which was tried and simply moves faster and falls. The policy has to be *trained* at the new bandwidth.

On this rebuilt plant the scripted controller of Section 3.8.13 was re-tuned against a full mission schedule: stand, walk forward, stand, walk backward, stand. It completes the mission in one 35 s episode on 3 of 3 seeds, at a drive-delivery ratio of 0.98 against 0.31 on the 0.8 Hz drive. The deliverable is small, but it is the first controller of any kind to demonstrate command-following locomotion on the plant the robot actually is. Tuning against the full mission mattered. A constant-velocity tune produced a walker that could not stand, which is the scripted twin of the RL command-curriculum lesson. **The m3 pitch wall survives the retune.** It is still about 1.0 s, still free fall, for both controller families, which confirms it as structural rather than bandwidth-limited.

A second, more ambitious scripted controller was built on the rebuilt plant: a capture-step state machine. It *failed*, and on the way down it measured four plant facts that any future controller inherits, learned or not. At 96 % of reach, commanded leg *extension* does not exist: a -65 mm vertical command moves the toe $+85$ mm *forward* and 8 mm down through the inverse kinematics, so weight support must be commanded in joint space rather than in task space. Capture steps land on air: the swung foot hovers 1 to 5 cm high through an entire fall, from rear-workspace inverse-kinematics error plus pitch lift, and a ground-seeking touchdown helps without sufficing. The 4-bar loop-closure constraint itself yields about 15 cm of ride height under single-leg load, with every actuated joint on target, and only a forward-kinematics height loop that includes the passive joints can see it. And the plant’s soft modes, measured by impulse response, put a weakly damped vertical body bounce near 1 Hz exactly where discrete stepping at 2.3 Hz pumps it. That last fact is the architectural reason the shuffling gait beats “smarter” stepping on this robot.

3.9 Negative results

Several substantial pieces of work in this project produced no usable artefact, and they are recorded here deliberately. Each cost real time. Each closes a door that would otherwise look open. In most cases the reason for the failure is more informative than a success would have been.

What was tried	Outcome	Why it is worth knowing
Direct joint-torque RL (Section 3.8.1)	Never converged; abandoned on a bandwidth argument that was later measured wrong	The action space was not the problem. The rejection premise was, and only instrumentation could tell.
Per-step position policy for walking (Section 3.8.2)	Skated, at 32 M steps, behind a healthy reward curve	Reward curves cannot certify a gait. Instrumented gait probes can.
Emulated actuated ankle (fixed torque reflex)	Strong version destabilised the gait; gentle version gave a small non-solving lift	A phase-blind torque fights the gait it stabilises. But the small positive identified the right lever.
Decaying external pitch assist, in four variants	Crutch dependence every time; the unaided policy was worse than no assist at all	Any aid that holds the robot up will be leaned on, and the in-training curve will hide it.
Lowering the simulated centre of mass by 6–12 cm	Slightly worse	Rules out the cheapest candidate plant fix.
Hard velocity, acceleration and cadence limits	Collapsed the warm-started policy	They were fighting a broken internal default (Section 3.8.6), not the gait.
Centre-of-mass identification from the excitation runs (Section 3.5.5)	Rejected: explains ≤ 11 –19 % of the gravity residual, 10 of 18 directions in the null space	The CAD centres of mass stand. Knowing the experiment cannot resolve them prevents fitting the same null space again.
Per-body mass, load-dependent stiction, base tilt, passive-pivot friction	All rejected as explanations of the same residual	The residual is conservative and gravity-correlated; no friction model can produce it.
Pure oscillator generator without a residual channel	Stability without locomotion: 15 m of 72, worst cadence in the set	Refutes the residual channel as the cause of the high-frequency footfall.

Table 3.18: Approaches that did not work, and what each one rules out.

3.10 Deployment

The policies of Section 3.8 ultimately have to run on the Raspberry Pi described in Section 3.2.4, against real drives, with no simulator to reset. Four questions decide whether that is viable. Can a non-realtime Linux close a control loop fast enough (Section 3.10.1)? Can the bus carry the traffic, and what happens when it cannot (Section 3.10.2)? What does the drive actually accept and report (Section 3.10.3)? And, the question this project learned the hard way, when something breaks on hardware, is the failure *investigable* (Section 3.10.5)? Measurement settled the first three. An incident settled the fourth.

3.10.1 Linux without a real-time kernel

The Pi runs a stock, non-realtime kernel. The loop-timing measurement of Section 3.5.6 established that the stock kernel suffices. A 200 Hz loop holds at half budget, with jitter well under a millisecond, and the drives broadcast at exactly 200 Hz anyway, so a faster loop would have nothing to read.

One Python-specific detail is load-bearing enough to record. CPython hands the interpreter lock

between threads at most every `switchinterval` seconds. The default is 5 ms, which is *exactly* one control period, so the web-interface thread could legally hold the interpreter across an entire control tick. Dropping the interval to 0.5 ms, with `sys.setswitchinterval(0.0005)`, took the daemon’s late ticks from roughly 10 % to a few percent. The steady state was then measured with the control daemon, the 200 Hz IMU thread, the web interface and the flight recorder of Section 3.10.5 all running. It is **188 Hz against a 200 Hz target, at a 5.30 ms mean period and 3 % late ticks**. The missing 6 % is operating-system scheduling, and no lever short of a realtime kernel recovers it.

An A/B under `systemd` overrides settled where the time actually goes. The daemon was restarted between arms, on an otherwise idle robot.

Configuration	Loop rate	Marginal cost
full stack (recorder + IMU + web UI)	186.7 Hz	—
recorder disabled	186.3 Hz	recorder: -0.01 ms/tick, i.e. free
recorder and sensor thread disabled	194.8 Hz	IMU thread: 0.23 ms/tick

Table 3.19: Component cost of the deployed stack, measured on the robot. The always-on flight recorder costs nothing measurable at 200 Hz; the 200 Hz IMU thread costs a twentieth of the budget.

A methodological trap, again. The first loop-rate figures taken during this bring-up read 60 Hz with 34 % late ticks. They produced a confident, and committed, diagnosis of an expensive failure path in the CAN layer. The diagnosis was wrong, because the A/B above shows the suspected path costs nothing. The 60 Hz was the *measurement disturbing the measured*. While the Pi was being sampled, `scp`, remote compilation, `journalctl` and HTTP polling were all running on its four cores. That is the same failure mode as the IMU noise figures taken on a swaying rig (Section 3.2.5): the number was real, the attribution was not. Two rules follow. Never benchmark the robot over the same SSH session that is deploying to it. And keep the retraction in the history rather than rewriting it.

Process supervision. The daemon runs under `systemd`. It starts at boot and restarts on crash, bounded to five restarts per minute so that a crash loop stays visibly down instead of power-cycling the motors. Its stdout goes to the journal. It is stopped by `SIGINT`, so that the shutdown path runs. That path streams zero current to every motor and writes the recorder’s clean-stop record. The *absence* of that record is how a power cut is later inferred (Section 3.10.5). Before supervision, the only record of the daemon’s death was whatever terminal happened to still be open.

3.10.2 CAN at runtime: the bus with nothing on it

Section 3.5.7 describes the protocol itself, and bandwidth is a non-issue at 26 % of one of two 1 Mbit buses. What the protocol description does not cover is the failure mode that running under `systemd` made routine: **the daemon now starts before the motor power is on**. With nothing on the bus to acknowledge a frame, `socketcan`’s ten-frame transmit queue fills within one tick, and every subsequent `send()` raises `ENOBUFS`. On its first boot under `systemd`, that exception went unhandled in a background thread and killed the control daemon outright, during its own preflight. The bug predates the service. Hand-started sessions had simply always had the motors powered first.

The fix treats a send failure as an *event*, *not a death*. Sends never raise past the CAN layer,

failures are counted per channel, and a channel that fails ten consecutive sends is retried at 5 Hz instead of 200. One successful send restores the full rate. Two honest measurements qualify the fix. First, a failing send costs nothing. With the backoff disabled and three sends per tick failing, the loop runs at 188.9 Hz, against 188.5 Hz with the backoff enabled. The backoff is kept for hygiene, since it removes 600 pointless system calls per second, and for forensics. “We did not attempt the frame” and “we attempted it and it failed” are different facts, and the recorder logs both. Second, safety does not rest on the backoff. Motors on a dead bus report nothing, so the motion interlocks refuse to command them. And a bus that starts refusing frames *while* the robot is being commanded trips the safety stop within half a second.

3.10.3 The drive as deployed

Three properties of the CubeMars drives dominate the deployment design. All three were measured rather than documented (Sections 3.5.3 and 3.5.7).

First, the only writable surface is the servo-mode command pair, `SET_POS` and `SET_CURRENT`. The internal cascaded gains are board configuration, readable through the vendor tool but not over CAN, and the position loop they implement is proportional-only with a 0.8 Hz corner.

Second, the drives report no bus voltage, so pack sag under load is unobservable from the drive side.

Third, and with the sharpest consequences, **the multi-turn position origin is not stable**. It re-randomises on every power cycle, and the incident below is consistent with it moving *between* power cycles as well. Every absolute position command is therefore only as good as a calibration captured earlier in the same session. Nothing in the drive signals when that calibration has silently become wrong.

3.10.4 The safety envelope as an instrument

A browser-based control interface runs on the robot, and it is the only path through which the hardware is commanded by hand. It is treated here as methodology rather than as tooling, because the safety envelope it enforces is also an experimental instrument.

- **Measured joint limits, not CAD limits.** Each joint is backdriven by hand through its full travel with the motors limp, and the excursion is recorded. The result is a per-joint range that reflects the assembled robot, including cabling and interference. The 4-bar’s assemblable band is a constraint on a joint *pair*, so the recorded reachable region is a box in the foot’s fore-aft and lift coordinates, not a set of independent joint ranges.
- **The recorded workspace is reused in simulation.** The same measured box is applied as a termination criterion during training. An episode ends the moment a foot leaves the region the real robot is known to reach. The criterion began as a safety measure and became a sim-to-sim gate, because it makes an entire class of simulator-only exploit impossible rather than merely expensive. It must also be applied carefully. It is expressed in the base frame, so by itself it cannot distinguish a collapsed ankle from a folded linkage (Section 3.4.3).
- **Playback is capped by kinematics, not by preference.** A part failed during a high-cadence trajectory playback, and the frequency ceiling was reduced from 5 Hz to 2.5 Hz. The new number was derived rather than guessed. Differentiating the stored waveform gives the peak joint speed per unit cadence, which is a property of the trajectory *shape*. The cam joint demands 359 deg/s per hertz, and reaches the actuator’s no-load speed at 3.5 Hz. Past that point the command is not merely hard to track but kinematically impossible, and the drive simply saturates current chasing a target it cannot reach. At the 5 Hz previously allowed, the

cam was being commanded at 143 % of no-load speed. The 2.5 Hz ceiling leaves it at 71 %. Raising the ceiling again requires a hardware argument. Because the limit depends on the shape, a gentler trajectory can safely go faster.

- **Trip conditions.** Tracking error, joint range and temperature are monitored continuously, and any of them latches the drives to zero current. The tracking-error trip in particular had to be decoupled from the commanded trajectory, once Section 3.5.3 showed that a large tracking error at speed is the *expected* behaviour of this drive rather than a fault.

3.10.5 Hardware validation: the flight recorder and the pre-move guard

On 2026-08-10 a joint destroyed itself on the bench. A zero pose had been captured, and by the time an absolute move was commanded, the drives' raw origin no longer matched it. The drive faithfully executed a position command in the wrong frame, and wound `left.cam` through roughly 1.9 output turns on a joint with $\pm 86^\circ$ of travel. Investigating it was *impossible*. The only telemetry in the system was a 4096-sample RAM ring at 20 Hz, of which the HTTP interface could serve the last 25.6 seconds, and nothing was written to disk. By the time anyone looked, the window had rolled past the event. The rest of this section describes what was built in response. "Make the failure investigable" turned out to be a design problem with measurable requirements, not a logging afterthought.

A three-tier flight recorder. Streaming the full state to disk continuously is not an option. Six motors at 200 Hz produce 35 kB/s, or 3 GB/day, which fills an SD card and then destroys it. The recorder therefore splits by rate.

Tier	Content	Mechanism
A	every field, continuously	20 Hz, fixed-width binary segments, rotated under a global disk budget
B	the black box proper: the full 200 Hz state	a 40 s RAM ring, dumped to its own file on any trigger, <i>pre-trigger history included</i> , plus a 3 s post-trigger tail
C	one JSON line per event	append-only, <code>fsync</code> 'd on write, never deleted

Table 3.20: The flight recorder. Tier B triggers include every safety trip, e-stop, mode transition, calibration or configuration write, warn-level tracking error (below the trip threshold, so the *approach* is captured, not only the fall), and an encoder-origin jump. Each dump embeds a content-hashed snapshot of the calibration and drive gains live at that instant, so a recording is self-describing without external context.

Two constraints shaped the implementation. First, the control loop must not pay for its own instrumentation. The 200 Hz thread only appends a tuple to a bounded queue, at about $2 \mu\text{s}$, and Table 3.19 confirms that the end-to-end cost is unmeasurable. A separate writer thread owns all file I/O. A full queue *drops and counts*, and the running drop count is stamped into the next accepted record, because a black box that cannot prove where its own gaps are is worse than none. Second, the robot must never depend on the recorder. If the writer thread dies, the control loop continues, and the interface surfaces the failure.

Time, without a trustworthy clock. The Pi has no real-time clock, and its access point has no internet, so the wall clock after boot is whatever `fake-hwclock` restored: plausible and wrong. Every record therefore carries four time fields. The monotonic clock is the primary

time base. The wall time comes with an explicit *trusted* flag, set only once NTP has actually synchronised. A per-boot kernel identifier and a per-run session identifier ensure that records from different sessions can never be silently interleaved. When NTP does land, the resulting step is itself logged, at +1333.5s on the first instrumented boot, so that earlier stamps can be corrected offline. Power-off is handled by inversion. A process cannot log its own death, so a heartbeat is `fsync`'d every 8s, and the last heartbeat before a gap is the power-off time, to heartbeat resolution. The offline reader states that inference explicitly rather than implying a clean shutdown.

The pre-move guard. The incident's mechanism was a full-authority slew against a stale calibration. Four mechanisms close it, and none of them adds a step to the normal workflow. That last clause is a requirement, not a nicety. An early draft forbade homing as the first move after calibration, and that was exactly backwards: after a zero capture the legs are limp and sagging, and homing is precisely what catches them.

1. **Hold before moving.** The first command after enabling is “hold the raw position you are reporting right now”, for 0.35s. It never consults the calibration, so it is safe even against a completely wrong zero, and it arrests the sag immediately. Only then does the slew to an absolute target begin.
2. **Raw-at-rest fingerprint.** The raw encoder pose of all six motors is recorded at every successful zero capture. Before leaving the limp state, the live raw pose is compared against that record. The comparison is decisive after a restart, because the origin re-randomises, so a calibration restored from disk is refused until the robot is re-zeroed. Within a live session a 200 Hz continuity watchdog is used instead. An origin *rewrite* moves the reported position while the drive reports no speed, and that signature is exactly how a rewrite is distinguished from the joint being moved by hand. A range-sanity check backstops both: a calibration implying 678° on an $\pm 88^\circ$ axis is refused outright.
3. **A homing-specific tracking threshold**, at 8° rather than the 25° playback value. A slow guided slew should track well, so a stale zero shows up as tracking error within the first few degrees of travel.
4. **A per-joint travel budget**, as the final backstop. A guided move that sweeps more than 1.3× the joint's own range without arriving is stopped. On 2026-08-10 that check alone would have cut the excursion from 1.9 output turns to under 0.6.

Acceptance. A hardware-free test reproduces the shape of the incident. The mock bus grew an `origin_shift` fault injection, which moves the reported *and* the commanded frame while the shaft stays put and reports zero speed. That is what a board renumbering its origin actually looks like, and it is what makes the fault invisible to any check that trusts reported velocity. The test captures a zero, shifts the origin and commands a home. It then asserts three things, all without a human opening a raw file. The guard must refuse, with both raw poses on record. The triggered dump must contain at least ten seconds of pre-trigger 200 Hz data. And the offline reader's `--postmortem` mode must state what happened: boot and power-loss times, the raw-pose comparison, commanded against reported position through the event, the configuration live at that instant, and whether the origin ever moved while the robot was limp and untouched.

What it found in its first hour. The recorder justified itself before the day ended. Its first boot on the robot captured its own deployment failure: the `ENOBUFFS` crash of Section 3.10.2, complete with traceback, which had previously been invisible. Its steady-state telemetry then established that the left-leg bus was electrically dead, with zero frames transmitted and all three

motors silent. That turned “the robot feels wrong” into a specific, timestamped fact. It also ended a longer-standing blindness. Nothing in the stack had ever measured the control loop’s actual rate, so a loop running at a third of its nominal frequency had been indistinguishable from a healthy one. The tick counter the recorder stamps into every sample is what made every figure in Table 3.19 measurable at all.

3.11 Hardware experiments

Everything in this section ran on the physical robot, through the web instrument of Section 3.10. Section 3.5 reports the heavy measurement results, which are only situated here.

In-air gait demonstration and teach-and-replay. The first hardware milestone was a hand-authored straight-walking gait, with the body clamped in the air and no balance required. A pure-NumPy gait generator streamed servo-mode position commands over CAN. Its safer successor is teach-and-replay. The motors are held limp by streaming zero current, a leg is backdriven by hand through the desired cycle, and takes are recorded per side. A pure-NumPy pipeline then resamples, phase-aligns, detects the left/right mirror, closes the loop with an FFT low-pass, and replays the result. Replay runs either as drive-side position commands, or through a software PID onto current commands with a hard ampere clamp as the torque limit. Replaying on the robot exposed a compliance problem that later measurements explained. At low proportional gain the leg drew only about 2 A against a 20 A allowance, and moved with stick-slip judder. The practical fix was to raise k_p , add k_i against the gravity lag, and take the derivative on target minus actual velocity. That was an early, unquantified encounter with the drive-bandwidth ceiling of Section 3.5.3.

Workspace calibration. The safe joint-space workspace was recorded by backdriving each leg through its physically assemblable region, then eroding the recorded polygon. It does triple duty, as the training-time constraint, the runtime watchdog and the offline deploy gate. Calibration on hardware corrected the model in one substantial way: **the cam is a full crank**. The CAD-guessed joint range of $\pm 86^\circ$ is wrong. Recorded sweeps span about 245° of cam rotation, and the 4-bar assembles at every angle. The calibrated zero pose also sits about 1° outside the eroded safe band, so the safety net refuses a naive hold-at-zero and lets a guided homing move through the physical-feasibility check instead (Section 3.10.5).

The identification campaign. Quasi-static and swept-sine excitations were captured at 200 Hz, through the measurement mode of the web instrument. They produced the plant corrections of Section 3.5: the known-mass K_t measurement, the 0.8 Hz drive Bode response, the gravity-model residual and its rejected explanations. The campaign also demonstrated the instrument itself. Runs are self-describing, because each capture embeds the calibration, the drive gains and the weighed masses. The excitation planner now solves amplitude against travel, no-load speed and predicted tracking error jointly. It does so because an early sweep e-stopped the run six seconds in, by asking the drive for more than its position loop can execute.

Sensor bench. The IMU characterisation of Section 3.2.5 was performed in place on the robot, with the provenance-labelling scheme described there. It covers noise, latency, filter behaviour, and the motion-gating methodology that its first wrong numbers forced.

The incident, and what it built. On 2026-08-10 a stale calibration met a full-authority absolute move and destroyed a joint (Section 3.10.5). That the investigation was impossible was itself the finding. It produced the flight recorder, the pre-move guard and the systemd

supervision of Section 3.10, all validated by reproducing the incident against the mock CAN bus. The recorder's first hour on hardware then found a daemon-killing crash at boot, caused by an unpowered CAN bus, and established that the left-leg bus was electrically dead. That is the robot's state as this is written: the right leg is fully functional under the 200 Hz stack, measured at 188 Hz, and the left leg awaits repair of the failed joint.

3.11.1 Static end-effector force map

The cam actuator reaches the knee through the pushrod 4-bar, so the transmission from motor torque to toe force is a strong function of posture. Whether the strong direction is the direction running needs had never been checked.

The constrained toe Jacobian differentiates the loop closure out, so it carries the linkage's transmission ratio. Statics against the actuator torque box then gives the feasible force set at every reachable toe position. Self-tests check loop closure against the stance keyframe, the analytic Jacobian against central differences, and a fast analytic ankle column against the full Jacobian.

Three findings follow. All three are statics-only, so they bound capability rather than predict behaviour.

- **The leg stands at 96 % of its reach**, with about 3 mm of further extension available anywhere near the stance position. Everything else follows from sitting that close to the leg-extension singularity.
- **Force there is 19:1 anisotropic**: 16.4 body weights straight down against 0.85 body weights horizontally. That is the same near-singular corner the speed-ceiling analysis found from the power side, seen now from the force side. It is a property of the fully extended pose rather than of the whole workspace. Over the running stance region the median tilted capacity recovers to 5.95 BW, because compressed postures rotate their strong axis.
- **The motors are not the binding constraint. The ankle is, by a factor of eight**. Over the stance region the actuators give a median vertical capacity of 4.30 BW, and 77 % of the region clears the 3.5 BW running requirement. The passive ankle gives a median of 0.42 BW, and **0.0 %** of the region clears the requirement.

The last point is a third independent route to the conclusion of Section 3.8.5 and Section 3.6, reached from statics with no learning and no dynamics. The linkage and the motor sizing are adequate to generous. The foot spring is the design defect.

4 Results

4.1 Hardware

The delivered hardware platform is a biped weighed at 15.14 kg, or 14.72 kg after the carbon-ankle rebuild (Section 3.8.14). It carries six quasi-direct-drive actuators (Table 3.1) and a 13S2P pack of 480 Wh rated to 250 A peak (Section 3.2.3). A Raspberry Pi 3B closes a 200 Hz control loop on stock Linux, at 3 % late ticks with half its budget spare (Tables 3.9 and 3.19). Two 1 Mbit CAN buses run at 26 % utilisation. The IMU is characterised to provenance-labelled noise figures (Table 3.5), and Table 3.7 tabulates every correction in the measured plant model. The capability analysis on the corrected plant places the robot at about 2.7 m/s credible burst and about 1.3 m/s thermally sustained (Section 3.6.6). The deployment stack — calibration, safety guards and flight recorder — runs on the robot under systemd, and it has survived and diagnosed its first real incident (Section 3.10).

4.2 RL experiments

Table 3.14 remains the headline. The milestone ladder culminates in a 6-DOF-free plant that runs 60 s without falling at 2.6 m/s, and in the cadence-fixed m7 at 2.71 m/s. The latter is about 70 % of the credible ceiling as it stood before the plant correction. The consolidated findings sit around that result.

- **Action space.** Direct torque never converged, and it was judged undeployable on premises that measurement later partly overturned (Section 3.8.1). Per-step position targets converged to standing but skated (Section 3.8.2). The Fourier CPG, re-parameterised every step and wrapped in reflexes and residuals, is the design that works (Section 3.8.3).
- **Generator A/B.** The oscillator CPG and the Fourier series tie by measurement, with complementary strengths. The oscillator produced the single best controller, and the only one that executes the sprint’s stop phase. It walls at the roll degree of freedom, where the Fourier generator passes (Section 3.8.10).
- **The pitch wall belongs to the plant.** RL and a 21-gain classical controller fail identically at about 1 s, which equals the analytic free-topple time across a $5\times$ gravity sweep (Section 3.8.13). Only a $12\times$ ankle stiffening breaks it in simulation (Section 3.8.5). It survives the hardware drive retune to 3 Hz (Section 3.8.14), which makes it structural rather than bandwidth-limited.
- **On the rebuilt plant,** with the carbon ankle, the 3 Hz drive and velocity caps, the scripted controller demonstrates the first full command-following mission: stand, walk forward, walk backward, stand again. It runs 35 s on 3 of 3 seeds, at a drive delivery of 0.98 (Section 3.8.14).
- **On the measured plant,** the sim-to-real lineage completes the descent to the real 0.8 Hz drive, with a $3\times$ score gain over its parent. Its diagnosis history is itself a result, covering the steering asymmetry, the domain-randomisation ablation and the command-range fit (Section 3.8.12).
- **The ankle question closed as a build decision.** Statics eliminate the as-built soft spring. Learning selects the compression-capable strut. Buckling arithmetic rejects the fabricated part in favour of carbon (Section 3.8.11).

5 Discussion

5.1 Successes

The pipeline exists end to end, and every stage has been validated at least once. CAD exports to a loop-closed MJCF that matches the measured workspace at 100 % of poses. A capability analysis independently reproduces both the high-cadence optimum and the ankle-stiffness threshold that RL discovered. An action-space design turned a non-converging problem into a reliably trainable one, and an independent generator A/B confirmed that twice. A curriculum mechanism — base-DOF release, with rigid rails and a constant observation and action shape — carries warm starts across plants. A measurement campaign corrected the plant, using instruments built on the robot itself. And a deployment stack proved the value of its flight recorder within an hour of installation. The standing policy on the fully free plant — 60 seconds, zero falls, 2.6 m/s — is a real result on a hard plant. The drivable lineage’s survival of the descent to the measured 0.8 Hz drive is the strongest sim-to-real evidence in the record.

5.2 Failures

Direct torque control failed twice over, on convergence and on assumed deployability. Abandoning it was itself partly wrong, because the measured premises now point back toward current-mode control with the PD closed on the Pi. Fully autonomous untethered running was not reached. The sprint policies observe privileged state, the drivable lineage is not yet robust, and no policy has run on the robot. The record ambition is dead on this chassis by measurement, at about 1.3 m/s sustained against a 4 m/s record pace. Three further failures stand out. The roll milestone remains the RL ladder’s weak link. The 6 Hz footfall chatter survives every post-hoc lever, and awaits a from-scratch retrain with cadence pressure applied from step zero. And the physical robot’s left leg was lost to a calibration failure before any policy reached it, which is the clearest possible demonstration that deployment infrastructure is not overhead.

5.3 Lessons learned

1. **Optimisers exploit objectives, and neural networks are not the culprit.** PPO gamed the reward by skating, penetrating the floor, shuttling and committing suicide by penalty. The scripted baseline’s coordinate-descent tuner gamed its own objective identically. Every objective needs adversarial review before its optimiser gets it.
2. **The stability-margin advantage of learning is conditional, and the conditions are measurable.** The theoretical case for RL on an unstable plant is that the hand-designed controller lies inside the policy class, so the learned optimum can only match or exceed it (Section 2.5.2). Three premises carry that argument, and this project paid none of them in full. The plant was not exact enough, because every parameter moved (Table 3.7). The reward did not price distance from the failure boundary; it priced survival and distance, so the optimiser parked on the boundary. And the training budget did not reach the asymptote, because 600 M steps per milestone is large rather than asymptotic. Hence the m2 result, where a margin existed and the learning did not find it. The generalisable diagnostic is that **seed variance is the margin measurement**. A figure of 20.3 ± 21.6 s against 5 of 5 at the episode cap is not a worse mean. It is bimodality, and a learning curve plotting the mean will not show it. At m3 the comparison is vacuous rather than adverse. With an empty feasible set, every controller drawn from it performs identically, and RL optimises over controllers, not over physics (Section 3.8.13).

3. **The useful question about a gait prior is which kind, not how much.** A shape prior fixes the trajectory and exposes its timing. Tuning it does not enlarge the reachable set, so it is safe only when the morphology has a natural template to copy. A structural prior constrains the class and leaves the member free, and its reachable set grows with the parameterisation. DASH-01 is the case that separates the two, because its cam turned out to be a full crank with no biological analogue. The cadence agreement between the learned optimum and the independent physics sweep is the evidence that the wider space was genuinely used (Section 2.5.1). The prior never vanished. It moved into the action space, and stayed hand-wired at the one channel where feedback must not be blind.
4. **Every assumed plant parameter measured later moved pessimistically:** mass by +18%, torque by −15%, drive bandwidth by −16×, plus friction and the ankle. Measure before training, not after. And when a measurement lands, apply it as *the* plant rather than as an optional variant, or the wrong default gets trained on.
5. **Simulation results need the same scepticism as rewards.** Soft joint limits, constraints inactive at settle, frame-relative safety checks, stale keyframes, a solver-version split, and an actuator-velocity cap that was simply never present each produced a clean, converged, wrong result. The last of those let a prized policy “balance” at 23 rad/s on a 22 rad/s motor. Self-tests, smoke gates or measurement caught each one.
6. **Evaluation hygiene is a first-class deliverable.** Six rules earned their place: evaluate with training aids forced off, after three separate deceptions by curves with a crutch; evaluate on a fixed task across curricula; use at least 20 seeded episodes; use paired seeds and survivor counts under bimodality; report peak rather than final under one-way curricula; and treat `curriculum.json` as ground truth on restore. Four separate evaluation bugs each nearly discarded or crowned the wrong policy.
7. **Curricula need three rules.** Gate the advance, not just the opening. Never gate a rescue ramp on the competence it is meant to rescue. And diff configurations against the last working run before theorising.
8. **A parameter range widened “to be non-binding” can become the operating point.** A neutral network output decodes to the middle of the range, which is what produced the 25 Hz gait-clock bug. A comment asserting an unexercised assumption also stays wrong for months.
9. **Hardware integration dominates, and instrumentation pays for itself.** The measurement instrument found the drive ceiling. The flight recorder found a daemon-killing crash and a dead bus within its first hour. And the trap of the measurement disturbing the measured was walked into twice: once with the IMU on a swaying rig, once with the loop rate under SSH load. The second time produced a committed wrong conclusion that only an A/B retracted.
10. **Progressive validation works.** The base-DOF ladder localised failures to single degrees of freedom. The toy problem debugged the deploy chain at one degree of freedom. Telemetry cross-checks and the workspace-as-safety-net made hardware sessions survivable, and the one incident occurred precisely in the gap the guard now closes.

6 Conclusion and Future Work

6.1 Summary of Findings

This thesis set out to apply reinforcement learning to a custom high-speed biped, and it delivered three things. The first is a working method: the modelling, training and curriculum stack that produces a 2.6 m/s never-falling runner on the free simulated plant. The second is a measured robot: every load-bearing parameter of DASH-01 replaced by a measurement, with stated uncertainty and stated un-resolvables. The third is an honest account of the distance between the two. That distance is not rhetorical. The measured robot is heavier and weaker than the model it trained against, and — decisively — an order of magnitude slower in actuation. Its corrected ceiling of about 2.7 m/s burst and 1.3 m/s sustained reclassifies the platform from record contender to demonstrator. The learning method itself is vindicated wherever it can be tested. It matches classical control, and is matched by it, exactly at the plant’s own limits, which is what a healthy learning stack should do.

6.2 Challenges and Limitations

The limits of the present work are recorded throughout, and summarised once here. No policy has yet run on the hardware, and export plus inference on the Pi remain the critical path. The sprint results rest on privileged observations. Identification left the link inertia tensors and the 0.9 N·m gravity residual unresolved, with the kinematic model as the remaining suspect. The actuator model lacks a coupled torque–speed envelope, which makes it optimistic exactly at high speed. The capability analysis’s corrected figures are a sensitivity propagation awaiting a full re-sweep. And the physical robot currently stands with one functional leg.

6.3 Suggestions for Future Research

For *this* robot, the record orders four priorities. First, ONNX export and a Pi inference loop, because everything else waits on them. Second, current-mode control with the position PD closed on the Pi at 200 Hz, replacing the 0.8 Hz drive loop that every measurement identifies as the binding constraint. Third, the carbon ankle strut. Fourth, the from-scratch cadence retrain.

For a *successor* robot, the measured record ranks the design levers.

1. **Bus voltage before torque.** 87 % of feasible gaits are limited at the cam’s peak-power corner. Speed scales linearly with no-load speed, and not at all with peak torque.
2. **Distal mass.** At -0.93 (m/s)/kg it carries nine times the leverage of torso mass. A carbon shin, plus a proximally mounted spring on a tendon — a true Achilles — recovers about 0.5 m/s for free.
3. **Re-proportion or replace the 4-bar.** Only 0.20 to 0.35 m of a 1.06 m geometric span is usable. The standing pose sits at 96 % of reach, where the toe-force ellipse is 19:1 anisotropic: strong vertically, and weak in exactly the propulsive direction. Target a flat transmission ratio over about 0.5 m, with stance at about 85 % of reach.
4. **Size the ankle for 3.5 body weights.** The present passive ankle delivers a median of 0.42 BW over the running-stance region, which is $8\times$ short, and it fails in positive mechanical feedback.
5. **Cool the cam motor.** Sustained speed is what a 100 m record is made of, and the gap between the 167 N·m demanded and the 55 N·m continuous rating is thermal.

6. **Treat torque-mode control as an architectural requirement from day one.** That is the v1 lesson, and it was paid for twice.

Finally, the validated toe-Jacobian force map and the capability sweep together make a *design search in simulation* the natural next use of the training cluster. Such a search would range over bus voltage, gear ratio, distal mass, linkage proportions and ankle stiffness. It would search over robots, rather than over controllers for a robot already known to be the constraint.

7 Appendix

7.1 logbook

- 2025-02-20** Creation of this report, rough layout of the idea in chapter
- 2025-02-25** Longitudinal kinematic model, research about markov modelling and cycle
- 2025-02-26** dynamic simulation creation, implemented a basic simulation in pybullet with keyboard navigation and parametric surface + wheel rolling down
- 2025-02-27** refining efficiency to take into account whole cycle, reading paper
- 2025-02-28** Writing Idea about improving efficiency through identified parameters
- 2025-03-04** Adding graph about car occupancy and real world driving needs + reading paper
- 2025-03-06** Brainstorming kinematic, reading PhD thesis on narrow track
- 2025-03-09** URDF is a PAIN to implement to due lack of tool, trying to define a kinematic model in python
- 2025-03-11** reworking part of chapter 2
- 2025-03-17** reading paper about cars and bike and limiting factor to adoption
- 2025-03-18** keep reading paper, also about narrow track vehicle
- 2025-03-20** trying to structure what I learned from the papers
- 2025-03-21** defining the know unknown about car adoption factor (human side)
- 2025-03-24** Clarifying the project goal
- 2025-03-27** draw figure, added citation
- 2025-03-28** Manual definition is an epic failure... Two way (createMultiBody -; Works but hard to read), Create body and add constraint don't work (rotation constraint crash the simulation.)
- 2025-04-09** Realized that createMultibody is more stable but has a "ghost" friction problem, Might as well use URDF then.
- 2025-04-10** learning URDF and how the origin is defined,
- 2025-05-25** redefined most variant as URDF and how to simulate them. Need spend time to implement basic controller like PID to show steady state stability and control margin
- 2025-05-28** Meeting, dry run presentation and shown current status.
- 2025-05-31** Report restructuring, writing
- 2025-06-02** Chapter 4, 5, 6 writing
- 2025-06-04** Getting simulation result, finishing first draft of report.
- 2025-06-08** Proof reading

7.2 URDF from CAD: step by step

References

- [1] Guillaume Bellegarda and Auke Ijspeert. CPG-RL: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 2022. arXiv:2211.00458.
- [2] Reinhard Blickhan. The spring-mass model for running and hopping. *Journal of Biomechanics*, 22(11–12):1217–1227, 1989.
- [3] Christine Chevallereau, Gabriel Abba, Yannick Aoustin, Franck Plestan, Eric R. Westervelt, Carlos Canudas-de Wit, and Jessy W. Grizzle. RABBIT: A testbed for advanced control theory. *IEEE Control Systems Magazine*, 23(5):57–79, 2003.
- [4] Steve Collins, Andy Ruina, Russ Tedrake, and Martijn Wisse. Efficient bipedal robots based on passive-dynamic walkers. *Science*, 307(5712):1082–1085, 2005.
- [5] Devin Crowley, Jeremy Dao, Helei Duan, Kevin Green, Jonathan Hurst, and Alan Fern. Optimizing bipedal locomotion for the 100m dash with comparison to human running. *arXiv preprint*, 2025. arXiv:2508.03070.
- [6] Jared Di Carlo, Patrick M. Wensing, Benjamin Katz, Gerardo Bledt, and Sangbae Kim. Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9, 2018.
- [7] Robert J. Full and Daniel E. Koditschek. Templates and anchors: Neuromechanical hypotheses of legged locomotion on land. *Journal of Experimental Biology*, 202(23):3325–3332, 1999.
- [8] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *arXiv preprint*, 2017. arXiv:1707.02286.
- [9] Kazuo Hirai, Masato Hirose, Yuji Haikawa, and Toru Takenaka. The development of Honda humanoid robot. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1321–1326, 1998.
- [10] Christian Hubicki, Jesse Grimes, Mikhail Jones, Daniel Renjewski, Alexander Sprowitz, Andy Abate, and Jonathan Hurst. ATRIAS: Design and validation of a tether-free 3D-capable spring-mass bipedal robot. *The International Journal of Robotics Research*, 35(12):1497–1521, 2016.
- [11] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- [12] Auke Jan Ijspeert. Central pattern generators for locomotion control in animals and robots: a review. *Neural Networks*, 21(4):642–653, 2008.
- [13] Atil Iscen, Ken Caluwaerts, Jie Tan, Tingnan Zhang, Erwin Coumans, Vikas Sindhwani, and Vincent Vanhoucke. Policies modulating trajectory generators. *Conference on Robot Learning (CoRL)*, 2018. arXiv:1910.02812.
- [14] Shuuji Kajita, Fumio Kanehiro, Kenji Kaneko, Kiyoshi Fujiwara, Kensuke Harada, Kazuhito Yokoi, and Hirohisa Hirukawa. Biped walking pattern generation by using preview control of zero-moment point. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1620–1626, 2003.

- [15] Shuuji Kajita, Fumio Kanehiro, Kenji Kaneko, Kazuhito Yokoi, and Hirohisa Hirukawa. The 3D linear inverted pendulum mode: A simple modeling for a biped walking pattern generation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 239–246, 2001.
- [16] Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Mini Cheetah: A platform for pushing the limits of dynamic quadruped control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6295–6301, 2019.
- [17] Scott Kuindersma, Robin Deits, Maurice Fallon, Andrés Valenzuela, Hongkai Dai, Frank Permenter, Twan Koolen, Pat Marion, and Russ Tedrake. Optimization-based locomotion planning, estimation, and control design for the Atlas humanoid robot. *Autonomous Robots*, 40(3):429–455, 2016.
- [18] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020.
- [19] Hun-Ok Lim and Atsuo Takanishi. Biped walking robots created at Waseda university: WL and WABIAN family. *Philosophical Transactions of the Royal Society A*, 365(1850):49–64, 2007.
- [20] Tad McGeer. Passive dynamic walking. *The International Journal of Robotics Research*, 9(2):62–82, 1990.
- [21] Takahiro Miki, Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.
- [22] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics (SIGGRAPH)*, 37(4), 2018.
- [23] Gill A. Pratt and Matthew M. Williamson. Series elastic actuators. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 399–406, 1995.
- [24] Jerry Pratt, John Carff, Sergey Drakunov, and Ambarish Goswami. Capture point: A step toward humanoid push recovery. In *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pages 200–207, 2006.
- [25] Ilija Radosavovic, Tete Xiao, Bike Zhang, Trevor Darrell, Jitendra Malik, and Koushil Sreenath. Real-world humanoid locomotion with reinforcement learning. *Science Robotics*, 9(89):eadi9579, 2024. arXiv:2303.03381.
- [26] Marc Raibert, Kevin Blankespoor, Gabriel Nelson, and Rob Playter. BigDog, the rough-terrain quadruped robot. In *IFAC Proceedings Volumes (17th IFAC World Congress)*, volume 41, pages 10822–10825, 2008.
- [27] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, 1986.
- [28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017. arXiv:1707.06347.
- [29] Sangok Seok, Albert Wang, Meng Yee (Michael) Chuah, Dong Jin Hyun, Jongwoo Lee, David M. Otten, Jeffrey H. Lang, and Sangbae Kim. Design principles for energy-efficient legged locomotion and implementation on the MIT cheetah robot. *IEEE/ASME Transactions on Mechatronics*, 20(3):1117–1129, 2015.
- [30] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all

- common bipedal gaits via periodic reward composition. *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. arXiv:2011.01387.
- [31] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems (RSS)*, 2018. arXiv:1804.10332.
- [32] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.
- [33] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [34] Miomir Vukobratović and Branislav Borovac. Zero-moment point — thirty five years of its life. *International Journal of Humanoid Robotics*, 1(1):157–173, 2004.
- [35] Miomir Vukobratović and Davor Juricic. Contribution to the synthesis of biped gait. *IEEE Transactions on Bio-Medical Engineering*, 16(1):1–6, 1969.
- [36] Patrick M. Wensing, Albert Wang, Sangok Seok, David Otten, Jeffrey Lang, and Sangbae Kim. Proprioceptive actuator design in the MIT cheetah: Impact mitigation and high-bandwidth physical interaction for dynamic legged robots. *IEEE Transactions on Robotics*, 33(3):509–522, 2017.
- [37] Eric R. Westervelt, Jessy W. Grizzle, Christine Chevallereau, Jun-Ho Choi, and Benjamin Morris. *Feedback Control of Dynamic Bipedal Robot Locomotion*. CRC Press, 2007.
- [38] Eric R. Westervelt, Jessy W. Grizzle, and Daniel E. Koditschek. Hybrid zero dynamics of planar biped walkers. *IEEE Transactions on Automatic Control*, 48(1):42–56, 2003.
- [39] Pierre-Brice Wieber. Trajectory free linear model predictive control for stable walking in the presence of strong perturbations. In *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pages 137–142, 2006.